

YARA-L 2.0 language syntax

Supported in: [Google secops](#) [SIEM](#) (/chronicle/docs/secops/google-secops-siem-toc)

This section describes the major elements of the YARA-L syntax. See also [Overview of the YARA-L 2.0 language](#) (/chronicle/docs/detection/yara-l-2-0-overview).

Note: YARA-L syntax doesn't allow negative integers. For example, `$e.principal.ip[-1]` is not valid. Replace `-1` with `0-1`.

Rule structure

For YARA-L 2.0, you must specify variable declarations, definitions, and usages in the following order:

1. meta
2. events
3. match (optional)
4. outcome (optional)
5. condition
6. options (optional)

Note: If you exclude `match`, the rule can match against a single event.

The following example illustrates the generic structure of a rule:

```
rule <rule Name>
{
    meta:
        // Stores arbitrary key-value pairs of rule details, such as who wrote
        // it, what it detects on, version control, etc.

    events:
        // Conditions to filter events and the relationship between events.
```

```
match:
    // Values to return when matches are found.

outcome:
    // Additional information extracted from each detection.

condition:
    // Condition to check events and the variables used to find matches.

options:
    // Options to turn on or off while executing this rule.
}
```

Meta section syntax

Meta section is composed of multiple lines, where each line defines a key-value pair. A key part must be an unquoted string, and a value part must be a quoted string:

```
<key> = "<value>"
```

The following is an example of a valid meta section line:

```
meta:
    author = "Google"
    severity = "HIGH"
```

Events section syntax

In the events section, list the predicates to specify the following:

- Variable declarations
- Event variable filters
- Event variable joins

Variable declarations

For variable declarations, use the following syntax:

- `<EVENT_FIELD> = <VAR>`
- `<VAR> = <EVENT_FIELD>`

Both are equivalent, as shown in the following examples:

- `$e.source.hostname = $hostname`
- `$userid = $e.principal.user.userid`

This declaration indicates that this variable represents the specified field for the event variable. When the event field is a repeated field, the match variable can represent any value in the array. It is also possible to assign multiple event fields to a single match or placeholder variable. This is a transitive join condition.

For example, the following:

- `$e1.source.ip = $ip`
- `$e2.target.ip = $ip`

Are equivalent to:

- `$e1.source.ip = $ip`
- `$e1.source.ip = $e2.target.ip`

When a variable is used, the variable must be declared through variable declaration. If a variable is used without any declaration, it is regarded as a compilation error.

Event variable filters

A boolean expression (#boolean_expressions) that acts on a single event variable is considered a filter.

Event variable joins

All event variables used in the rule must be joined with every other event variable in either of the following ways:

- Directly through an equality comparison between event fields of the two joined event variables, for example: `$e1.field = $e2.field`. The expression must not include arithmetic.

- Indirectly through a transitive join involving only an event field (see [variable declaration](#) (#variable_declarations) for a definition of "transitive join"). The expression must not include arithmetic.

For example, assuming \$e1, \$e2, and \$e3 are used in the rule, the following events sections are valid.

```
events:
  $e1.principal.hostname = $e2.src.hostname // $e1 joins with $e2
  $e2.principal.ip = $e3.src.ip // $e2 joins with $e3
```

```
events:
  // $e1 joins with $e2 via function to event comparison
  re.capture($e1.src.hostname, ".*") = $e2.target.hostname
```

```
events:
  // $e1 joins with $e2 via an `or` expression
  $e1.principal.hostname = $e2.src.hostname
  or $e1.principal.hostname = $e2.target.hostname
  or $e1.principal.hostname = $e2.principal.hostname
```

```
events:
  // all of $e1, $e2 and $e3 are transitively joined via the placeholder variable
  $e1.src.ip = $ip
  $e2.target.ip = $ip
  $e3.about.ip = $ip
```

```
events:
  // $e1 and $e2 are transitively joined via function to event comparison
  re.capture($e2.principal.application, ".*") = $app
  $e1.principal.hostname = $app
```

Note: If your sole join condition is an **or** chain, a function to event comparison, or a combination of both, then the rule may perform poorly.

However, here are examples of invalid **events** sections.

```
events:
  // Event to arithmetic comparison is an invalid join condition for $e1 and $e2
  $e1.principal.port = $e2.src.port + 1
```

```
events:
  $e1.src.ip = $ip
  $e2.target.ip = $ip
  $e3.about.ip = "192.1.2.0" //$e3 is not joined with $e1 or $e2.
```

```
events:
  $e1.src.port = $port

  // Arithmetic to placeholder comparison is an invalid transitive join condition
  $e2.principal.port + 800 = $port
```

Match section syntax

In the **match** section, list the match variables for group events before checking for match conditions. Those fields are returned with each match.

- Specify what each match variable represents in the **events** section.
- Specify the time duration to use to correlate events after the **over** keyword. Events outside the time duration are ignored.
- Use the following syntax to specify the time duration: **<number><m/h/d>**

Where **m/h/d** means minutes, hours, and days respectively.

- Minimum time you can specify is 1 minute.
- Maximum time you can specify is 48 hours.

The following is an example of a valid `match`:

```
$var1, $var2 over 5m
```

This statement returns `$var1` and `$var2` (defined in the `events` section) when the rule finds a match. The time specified is 5 minutes. Events that are more than 5 minutes apart are not correlated and therefore ignored by the rule.

Here is another example of a valid `match` section:

```
$user over 1h
```

This statement returns `$user` when the rule finds a match. The time window specified is 1 hour. Events that are more than an hour apart are not correlated. The rule does not consider them to be a detection.

Here is another example of a valid `match` section:

```
$source_ip, $target_ip, $hostname over 2m
```

This statement returns `$source_ip`, `$target_ip`, and `$hostname` when the rule finds a match. The time window specified is 2 minutes. Events that are more than 2 minutes apart are not correlated. The rule does not consider them to be a detection.

The following examples illustrate **invalid** `match` sections:

- `var1, var2 over 5m // invalid variable name`
- `$user 1h // missing keyword`

Zero value handling in the match section

Rules Engine implicitly filters out the zero values for all placeholders that are used in the match section ("" for string, 0 for numbers, false for booleans, the value in position 0 for enumerated types (/chronicle/docs/reference/udm-field-list#event_enumerated_types)). The following example illustrates rules that filter out the zero values.

```
rule ZeroValuePlaceholderExample {
  meta:
  events:
    // Because $host is used in the match section, the rule behaves
    // as if the following predicate was added to the events section:
    // $host != ""
    $host = $e.principal.hostname

    // Because $otherPlaceholder was not used in the match section,
    // there is no implicit filtering of zero values for $otherPlaceholder.
    $otherPlaceholder = $e.principal.ip

  match:
    $host over 5m

  condition:
    $e
}
```

However, if a placeholder is assigned to a function, rules don't implicitly filter out the zero values of placeholders that are used in the match section. The following example illustrates rules that filter out the zero values:

```
rule ZeroValueFunctionPlaceholder {
  meta:
  events:
    // Even though $ph is used in the match section, there is no
    // implicit filtering of zero values for $ph, because $ph is assigned to :
    $ph = re.capture($e.principal.hostname, "some-regex")

  match:
    $ph over 5m

  condition:
    $e
}
```

To disable the implicit filtering of zero values, you can use the `allow_zero_values` option in the [options section](#) (`#options_section_syntax`).

Hop window

By default, YARA-L 2.0 rules with a `match` section are evaluated using hop windows. The time range of the rule's execution is divided into a set of overlapping hop windows, each with the duration specified in the `match` section. Events are then correlated within each hop window.

For example, for a rule that is run over the time range [1:00, 2:00], with a `match` section over `30m`, a possible set of overlapping hop windows that could be generated is [1:00, 1:30], [1:03, 1:33] and [1:06, 1:36]. These windows are used to correlate multiple events.

Sliding window

Using hop windows is not an effective way to search for events that happen in a specific order (for example, `e1` happens up to 2 minutes after `e2`). An occurrence of event `e1` and an occurrence of event `e2` are correlated only if they fall into the same hop window generated.

A more effective way to search for such event sequences is to use sliding windows. Sliding windows with the duration specified in the `match` section are generated when beginning or ending with a specified pivot event variable. Events are then correlated within each sliding window. This makes it possible to search for events that happen in a specific order (for example, `e1` happens within 2 minutes of `e2`). An occurrence of event `e1` and an occurrence of event `e2` are correlated if event `e1` occurs within the sliding window duration after event `e2`.

Specify sliding windows in the `match` section of a rule as follows:

```
<match-var-1>, <match-var-2>, ... over <duration> before|after <pivot-event-var>
```

The pivot event variable is the event variable that sliding windows are based on. If you use the `before` keyword, sliding windows are generated, ending with each occurrence of the pivot event. If the `after` keyword is used, sliding windows are generated beginning with each occurrence of the pivot event.

The following are examples of valid sliding window usages:

- `$var1, $var2 over 5m after $e1`
- `$user over 1h before $e2`

See a sliding window rule example

([/chronicle/docs/detection/yara-l-2-0-overview#sliding_window_rule_example](https://chronicle/docs/detection/yara-l-2-0-overview#sliding_window_rule_example)).

Note: Using sliding windows instead of hop windows has been known to result in slower performance. We recommend using sliding windows only for specific cases, such as when event order is absolutely necessary or when searching for the non-existence of events.

We recommend not using sliding windows for single-event rules, because sliding windows are designed to detect multiple events. If one of your rules falls in this category, We recommend one of the following workarounds:

- Convert the rule to use multiple event variables, and update the condition section if the rule requires more than one occurrence of the event.
 - Optionally, consider adding timestamp filters instead of using a sliding window. For example, `$permission_change.metadata.event_timestamp.seconds < $file_creation.metadata.event_timestamp.seconds`
- Remove the sliding window.

Outcome section syntax

In the `outcome` section, you can define up to 20 outcome variables, with arbitrary names. These outcomes will be stored in the detections generated by the rule. Each detection may have different values for the outcomes.

The outcome name, `$risk_score`, is special. You can optionally define an outcome with this name, and if you do, it must be an integer or float type. If populated, the `risk_score` will be shown in the Enterprise Insights view

(<https://cloud.google.com/chronicle/docs/investigation/view-alerts-insights.md>) for alerts that come from rule detections.

If you don't include a `$risk_score` variable in the outcome section of a rule, one of the following default values is set:

- If the rule is configured to generate an alert, then `$risk_score` is set to 40.
- If the rule is not configured to generate an alert, then `$risk_score` is set to 15.

The value of `$risk_score` is stored in the `security_result.risk_score` UDM field.

Outcome variable data types

Each outcome variable can have a different data type, which is determined by the expression used to compute it. We support the following outcome data types:

- integer
- floats
- string
- lists of integers
- lists of floats
- lists of strings

Conditional logic

You can use conditional logic to compute the value of an outcome. Conditionals are specified using the following syntax pattern:

```
if(BOOL_CLAUSE, THEN_CLAUSE)  
if(BOOL_CLAUSE, THEN_CLAUSE, ELSE_CLAUSE)
```

You can read a conditional expression as "if BOOL_CLAUSE is true, then return THEN_CLAUSE, else return ELSE_CLAUSE".

BOOL_CLAUSE must evaluate to a boolean value. A BOOL_CLAUSE expression takes a similar form as expressions in the `events` section. For example, it can contain:

- UDM field names with comparison operator, for example:

```
if($context.graph.entity.user.title = "Vendor", 100, 0)
```

- placeholder variable that was defined in the `events` section, for example:

```
if($severity = "HIGH", 100, 0)
```

- another outcome variable defined in the `outcome` section, for example:

```
if($risk_score > 20, "HIGH", "LOW")
```

- functions that return a boolean, for example:

```
if(re.regex($e.network.email.from, `.*altostrat.com`), 100, 0)
```

- look up in a reference list (#reference_lists_syntax), for example:

```
if($u.principal.hostname in %my_reference_list_name, 100, 0)
```

- aggregation comparison, for example:

```
if(count($login.metadata.event_timestamp.seconds) > 5, 100, 0)
```

The THEN_CLAUSE and ELSE_CLAUSE must be the same data type. We support integers, floats, and strings.

You can omit the ELSE_CLAUSE if the data type is integer or a float. If omitted, the ELSE_CLAUSE evaluates to 0. For example:

```
`if($e.field = "a", 5)` is equivalent to `if($e.field = "a", 5, 0)`
```

You must provide the ELSE_CLAUSE if the data type is string or if the THEN_CLAUSE is a placeholder variable or outcome variable.

Mathematical operations

You can use mathematical operations to compute integer or float data type in the `outcome` and `events` sections of a rule. Google Security Operations supports addition, subtraction, multiplication, division, and modulus as top level operators in a computation.

The following snippet is an example computation in the `outcome` section:

```
outcome:
  $risk_score = max(100 + if($severity = "HIGH", 10, 5) - if($severity = "LOW"
```

Mathematical operations are allowed on the following types of operands as long as each operand and the entire arithmetic expression is properly aggregated (See Aggregations (#aggregations)):

- Numeric event fields
- Numeric placeholder variables defined in the `events` section
- Numeric outcome variables defined in the `outcome` section

- Functions returning ints or floats
- Aggregations returning ints or floats

Modulus is not allowed on floats.

Placeholder variables in outcomes

When computing outcome variables, you can use placeholder variables which were defined in the events section of your rule. In this example, assume that `$email_sent_bytes` was defined in the events section of the rule:

Single-event example:

```
// No match section, so this is a single-event rule.

outcome:
  // Use placeholder directly as an outcome value.
  $my_outcome = $email_sent_bytes

  // Use placeholder in a conditional.
  $other_outcome = if($file_size > 1024, "SEVERE", "MODERATE")

condition:
  $e
```

Multi-event example:

```
match:
  // This is a multi event rule with a match section.
  $hostname over 5m

outcome:
  // Use placeholder directly in an aggregation function.
  $max_email_size = max($email_sent_bytes)

  // Use placeholder in a mathematical computation.
  $total_bytes_exfiltrated = sum(
    1024
    + $email_sent_bytes
    + $file_event.principal.file.size
  )
```

```
condition:  
  $email_event and $file_event
```

Outcome variables in outcome assignment expressions

Outcome variables can be used to derive other outcome variables, similar to placeholder variables defined in the `events` section. You can refer to an outcome variable in the assignment of another outcome variable with a `$` token followed by the variable name. Outcome variables must be defined before they can be referenced in the rule text. When used in an assignment expression, outcome variables must not be aggregated (See [Aggregations](#) (`#aggregations`)).

In the following example, the outcome variable `$risk_score` derives its value from the outcome variable `$event_count`:

Multi-event example:

```
match:  
  // This is a multi event rule with a match section.  
  $hostname over 5m  
  
outcome:  
  // Aggregates all timestamp on login events in the 5 minute match window.  
  $event_count = count($login.metadata.event_timestamp.seconds)  
  
  // $event_count cannot be aggregated again.  
  $risk_score = if($event_count > 5, "SEVERE", "MODERATE")  
  
  // This is the equivalent of the 2 outcomes above combined.  
  $risk_score2 = if(count($login.metadata.event_timestamp.seconds) > 5, "SEVERE", "MODERATE")  
  
condition:  
  $event_count > 5 and $risk_score == "SEVERE"
```

Outcome variables can be used in any type of expression on the right-hand-side of an outcome assignment, except in the following expressions:

- Aggregations
- `Arrays.length()` function calls

- With `any` or `all` modifiers

Aggregations

Repeated event fields are non-scalar values. That is, a single variable points to multiple values. For example, the event field variable `$e.target.ip` is a repeated field and can have zero, one, or many ip values. It is a non-scalar value. Whereas the event field variable `$e.principal.hostname` is not a repeated field and only has 1 value (i.e. a scalar value).

Similarly, both non-repeated event fields and repeated event fields used in the outcome section of a rule with a match window are non-scalar values. For example, the following rule groups events using a match section and refers to a non-repeated event field in the outcome section:

```
rule OutcomeAndMatchWindow{
  ...
  match:
    $userid over 5m
  outcome:
    $hostnames = array($e.principal.hostname)
  ...
}
```

Any 5-minute window the rule executes over might contain zero, one, or many events. The outcome section operates on all events in a match window. Any event field variable referred to within the outcome section can point to zero, one, or many values of the field on each event in the match window. For example, if a 5-minute window contains 5 `$e` events, `$e.principal.hostname` in the outcome section points to five different hostnames. The event field variable `$e.principal.hostname` is treated as a non-scalar value in the outcome section of this rule.

Because outcome variables must always yield a single scalar value, any non-scalar value which an outcome assignment depends on must be aggregated to yield a single scalar value. In an outcome section, the following are non-scalar values and must be aggregated:

- Event fields (repeated or non-repeated) when the rule uses a match section
- Event placeholders (repeated or non-repeated) when the rule uses a match section
- Repeated event fields when the rule does not use a match section
- Repeated event placeholders when the rule does not use a match section

Scalar event fields, scalar event placeholders, and constants can be wrapped in aggregation functions in rules that don't include a match section. However, in most cases, these aggregations return the wrapped value, making them unnecessary. An exception is the `array()` aggregation, which you can use to explicitly convert a scalar value into an array.

Outcome variables are treated like aggregations: they must not be re-aggregated when referred to in another outcome assignment.

You can use the following aggregation functions:

- `max()`: outputs the maximum over all possible values. Only works with integer and float.
- `min()`: outputs the minimum over all possible values. Only works with integer and float.
- `sum()`: outputs the sum over all possible values. Only works with integer and float.
- `count_distinct()`: collects all possible values, then outputs the distinct count of possible values.
- `count()`: behaves like `count_distinct()`, but returns a non-distinct count of possible values.
- `array_distinct()`: collects all possible distinct values, then outputs a list of these values. It truncates the list of distinct values to 1,000 random elements. The deduplication to get a distinct list is applied first, then the truncation is applied.
- `array()`: behaves like `array_distinct()`, but returns a non-distinct list of values. It also truncates the list of values to 1,000 random elements.
- `period_start_for_max()`: start of the time period where the maximum of the listed value occurred.
- `period_start_for_min()`: start of the time period where the minimum of the listed value occurred.

The aggregate function is important when a rule includes a `condition` section that specifies multiple events must exist, because the aggregate function will operate on all the events that generated the detection.

For example, if your `outcome` and `condition` sections contain:

```
outcome:
  $asset_id_count = count($event.principal.asset_id)
```

```
$asset_id_distinct_count = count_distinct($event.principal.asset_id)

$asset_id_list = array($event.principal.asset_id)
$asset_id_distinct_list = array_distinct($event.principal.asset_id)
```

```
condition:
  #event > 1
```

Since the condition section requires there to be more than one event for each detection, the aggregate functions will operate on multiple events. Suppose the following events generated one detection:

```
event:
  // UDM event 1
  asset_id="asset-a"

event:
  // UDM event 2
  asset_id="asset-b"

event:
  // UDM event 3
  asset_id="asset-b"
```

Then the values of your outcomes will be:

- `$asset_id_count = 3`
- `$asset_id_distinct_count = 2`
- `$asset_id_list = ["asset-a", "asset-b", "asset-b"]`
- `$asset_id_distinct_list = ["asset-a", "asset-b"]`

Things to know when using the outcome section:

Other notes and restrictions:

- The **outcome** section cannot reference a new placeholder variable which wasn't already defined in the **events** section or in the **outcome** section.
- The **outcome** section cannot use event variables that have not been defined in the **events** section.

- The **outcome** section can use an event field that was not used in the **events** section, given that the event variable that the event field belongs to was already defined in the **events** section.
- The **outcome** section can only correlate event variables that have already been correlated in the **events** section. Correlations happen when two event fields from different event variables are equated.

You can find an example using the outcome section in [Overview of the YARA-L 2.0 \(/chronicle/docs/detection/yara-l-2-0-overview#rule_with_outcome_section_example\)](/chronicle/docs/detection/yara-l-2-0-overview#rule_with_outcome_section_example). See [Create context-aware analytics \(/chronicle/docs/detection/context-aware-analytics#outcome_section\)](/chronicle/docs/detection/context-aware-analytics#outcome_section) for details on detection deduping with the outcome section.

Condition section syntax

- specify a match condition over events and placeholders defined in the **events** section. See the following section, *Event and placeholder conditionals*, for more details.
- (optional) use the **and** keyword to specify a match condition using outcome variables defined in the **outcome** section. See the following section, *Outcome conditionals*, for more details.

Count character

The **#** character is a special character in the **condition** section. If it is used before any event or placeholder variable name, it represents the number of distinct events or values that satisfy all of the **events** section conditions.

For example, **#c > 1** means the variable **c** must occur more than 1 time.

Value character

The **\$** character is a special character in the **condition** section. If it is used before any outcome variable name, it represents the value of that outcome.

If it is used before any event or placeholder variable name (for example, **\$event**), it represents **#event > 0**.

Event and placeholder conditionals

List condition predicates for events and placeholder variables here.

You can join these predicates using the keyword **and** or **or**.

- Use the keyword **and** between any conditions.
- Use the keyword **or** only when the rule contains a single event variable.

A valid example of using **or** between two placeholders on the same event:

```
rule ValidConditionOr {
  meta:
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"

    // Note that all placeholders use the same event variable.
    $ph = $e.principal.user.userid // Define a placeholder variable to put
    $ph2 = $e.principal.ip // Define a second placeholder variable to put :
    $ph3 = $e.principal.hostname // Define a third placeholder variable to

  match:
    $ph over 5m

  condition:
    $ph2 or $ph3
}
```

An invalid example of using **or** between two conditions on different events:

```
rule InvalidConditionOr {
  meta:
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e2.graph.metadata.entity_type = "FILE"
    $e2.graph.entity.hostname = $e.principal.hostname

    $ph = $e.principal.user.userid // Define a placeholder variable to put

  match:
    $ph over 5m

  condition:
    $e or $e2 // This line will cause an error because there is an or between
```

```
}
```

Note: Don't use the keyword **not** in event and placeholder conditionals.

Bounded and Unbounded conditions

The following conditions are bounded conditions. They force the associated event variable to exist, meaning that at least one occurrence of the event must appear in any detection.

- `$var // equivalent to #var > 0`
- `#var > n // where n >= 0`
- `#var >= m // where m > 0`

The following conditions are unbounded conditions. They allow the associated event variable to not exist, meaning that it is possible that no occurrence of the event appears in a detection and any reference to fields on the event variable will yield a zero value.

Unbounded conditions can be used to detect the absence of an event over a period of time. For example, a threat event without a mitigation event within a 10 minute window. Rules using unbounded conditions are called non-existence rules.

- `!$var // equivalent to #var = 0`
- `#var >= 0`
- `#var < n // where n > 0`
- `#var <= m // where m >= 0`

Note: For non-existence rules, the detection engine adds a 1 hour delay to the expected latency (based on the rule's run frequency) to allow for late-arriving data.

Requirements for non-existence

For a rule with non-existence to compile, it must satisfy the following requirements:

1. At least one UDM event must have a bounded condition (that is, at least one UDM event must exist).

2. If a placeholder has an unbounded condition, it must be associated with at least one bounded UDM event.
3. If an entity has an unbounded condition, it must be associated with at least one bounded UDM event.

Consider the following rule with the condition section omitted:

```
rule NonexistenceExample {
  meta:
  events:
    $u1.metadata.event_type = "NETWORK_CONNECTION" // $u1 is a UDM event.
    $u2.metadata.event_type = "NETWORK_CONNECTION" // $u2 is a UDM event.
    $e1.graph.metadata.entity_type = "FILE"          // $e1 is an Entity.
    $e2.graph.metadata.entity_type = "FILE"          // $e2 is an Entity.

    $user = $u1.principal.user.userid // Match variable is required for Mul-

    // Placeholder Associations:
    //   u1           u2
    //   | \         /
    // port   ip
    //   |         \
    //   e1           e2
    $u1.target.port = $port
    $e1.graph.entity.port = $port
    $u1.principal.ip = $ip
    $u2.target.ip = $ip
    $e2.graph.entity.ip = $ip

    // UDM-Entity Associations:
    // u1 - u2
    // | \ |
    // e1  e2
    $u1.metadata.event_type = $u2.metadata.event_type
    $e1.graph.entity.hostname = $u1.principal.hostname
    $e2.graph.entity.hostname = $u1.target.hostname
    $e2.graph.entity.hostname = $u2.principal.hostname

  match:
    $user over 5m

  condition:
    <condition_section>
}
```

The following are *valid* examples for the `<condition_section>`:

- `$u1 and !$u2 and $e1 and $e2`
 - All UDM events and entities are present in the condition section.
 - At least one UDM event is bounded.
- `$u1 and !$u2 and $e1 and !$e2`
 - `$e2` is unbounded, which is allowed because it is associated with `$u1`, which is bounded. If `$e2` was not associated with `$u1`, this would be invalid.
- `#port > 50 and #ip = 0`
 - No UDM events and entities are present in the condition section; however, the placeholders that are present cover all the UDM events and entities.
 - `$ip` is assigned to both `$u1` and `$u2` and `#ip = 0` is an unbounded condition. However, bounded conditions are *stronger* than unbounded conditions. Since `$port` is assigned to `$u1` and `#port > 50` is a bounded condition, `$u1` is still bounded.

The following are *invalid* examples for the `<condition_section>`:

- `$u1 and $e1`
 - Every UDM event and entity appearing in the Events Section must appear in the Condition Section (or have a placeholder assigned to it that appears in the Condition Section).
- `$u1, $u2, $e1, $u2, #port > 50`
 - Commas are not allowed as condition separators.
- `!$u1 and !$u2 and $e1 and $e2`
 - Violates the first requirement that at least one UDM event is bounded.
- `($u1 or #port < 50) and $u2 and $e1 and $e2`
 - `or` keyword is not supported with unbounded conditions.
- `($u1 or $u2) and $e1 and $e2`
 - `or` keyword is not supported between different event variables.
- `not $u1 and $u2 and $e1 and $e2`
 - `not` keyword is not allowed for event and placeholder conditions.

- `#port < 50 and #ip = 0`
 - The placeholders that are present cover all the UDM events and entities; however, all of the conditions are unbounded. This means none of the UDM events are bounded, causing the rule to fail to compile.

Note: Don't use a `match` variable in the `condition` section. It is a semantic error since events are grouped by the `match` variable value.

Note: Don't specify only **unbounded conditions** on all `event` variables that a `match` variable is assigned to. It is a semantic error. For a `match` variable value to be returned, at least one event must exist that contains the value.

Note: In case of using a sliding window, the pivot event variable must be involved in at least one bounded condition.

Outcome conditionals

List condition predicates for outcome variables here, joined with the keyword `and` or `or`, or preceded by the keyword `not`.

Specify outcome conditionals differently depending on the type of the outcome variable:

- **integer:** compare against an integer literal with operators `=`, `>`, `>=`, `<`, `<=`, `!=`, for example:

```
$risk_score > 10
```

- **float:** compare against a float literal with operators `=`, `>`, `>=`, `<`, `<=`, `!=`, for example:

```
$risk_score <= 5.5
```

- **string:** compare against a string literal with either `=` or `!=`, for example:

```
$severity = "HIGH"
```

- **list of integers or arrays:** specify condition using the `arrays.contains` function, for example:

```
arrays.contains($event_ids, "id_1234")
```

Note: If you use the keyword **or** inside the <event/placeholder conditionals> subsection, you must surround that entire subsection with parentheses. For example, the following is valid: **(\$e1 or \$e2) and \$outcome > 0**.

Rule classification

Specifying an outcome conditional *in a rule that has a match section* means that the rule will be classified as a **multi-event** rule for rule quota. See [single event rule](/chronicle/docs/detection/yara-l-2-0-overview#single_event_rule) (/chronicle/docs/detection/yara-l-2-0-overview#single_event_rule) and [multiple event rule](/chronicle/docs/detection/yara-l-2-0-overview#multiple_event_rule) (/chronicle/docs/detection/yara-l-2-0-overview#multiple_event_rule) for more information about single and multiple event classifications.

Options section syntax

In the **options** section, you can specify the options for the rule. Here is an example of how to specify the options section:

```
rule RuleOptionsExample {  
  // Other rule sections  
  
  options:  
    allow_zero_values = true  
}
```

You can specify options using the syntax **key = value**, where **key** must be a predefined option name and **value** must be a valid value for the option, as specified for the following options:

allow_zero_values

The valid values for this option are **true** and **false**, which determine if this option is enabled or not. The default value is **false**. This option is disabled if it is not specified in the rule.

To enable this setting, add the following to the options section of your rule:

allow_zero_values = true. Doing so will prevent the rule from implicitly filtering out the

zero values of placeholders that are used in the match section, as described in [zero value handling in the match section](#) ([#zero_value_handling_in_the_match_section](#)).

suppression_window

The `suppression_window` option lets you control how often a rule triggers a detection. It prevents the same rule from generating multiple detections within a specified time window, even if the rule's conditions are met multiple times. Suppression windowing uses a tumbling window approach, which suppresses duplicates over a fixed-size, non-overlapping window.

You can optionally provide a `suppression_key` to further refine which instances of the rule are suppressed within the suppression window. If not specified, all instances of the rule are suppressed. This key is defined as an outcome variable.

In the following example, `suppression_window` is set to `5m` and `suppression_key` is set to the `$hostname` variable. After the rule triggers a detection for `$hostname`, any further detections for `$hostname` are suppressed for the next five minutes. However, if the rule triggers on an event with a different hostname, a detection is created.

The default value of `suppression_window` is `0`; that is, the suppression window is disabled by default. This option only works for [single event rules](#) ([/chronicle/docs/detection/yara-l-2-0-overview#single-event-rule](#)) that don't have a `match` section.

Example:

```
rule SuppressionWindowExample {
  // Other rule sections

  outcome:
    $suppression_key = $hostname

  options:
    suppression_window = 5m
}
```

Composite detection rules

Composite detection in Google SecOps involves connecting multiple YARA-L rules. This section explains how to build a composite rule. For an overview of composite detections, see [Overview of composite detections](/chronicle/docs/detection/composite-detections) (/chronicle/docs/detection/composite-detections).

Understand the rule structure

Composite detection rules are always multi-event rules and follow the same [structure and syntax](/chronicle/docs/detection/yara-l-2-0-syntax#rule_structure) (/chronicle/docs/detection/yara-l-2-0-syntax#rule_structure).

A composite rule has the following essential components:

1. **events** block: defines your inputs; the specific detections or events that the rule analyzes.
2. **match** block: specifies how the inputs should be connected over a defined time window.
3. **condition** block: contains the final logic that determines whether the joined events meet the criteria to trigger an alert.

Define the inputs in the events block

The first step is to define the rule's inputs in the **events** block. Inputs for composite rules come from [collections](/chronicle/docs/reference/rest/v1alpha/Collection) (/chronicle/docs/reference/rest/v1alpha/Collection), which store the detections generated by other rules. Google SecOps provides the following two methods to access data from collections.

Reference detection content using outcome variables, match variables, or meta labels

To access data from a detection without referencing the original UDM events, use **outcome** variables, **match** variables, or **meta** labels. We recommend this approach because it provides greater flexibility and better compatibility across different rule types.

For example, multiple rules can store a string (such as a URL, filename, or registry key) in a common **outcome** variable if you're looking for that string across different contexts. To access this string from a composite rule, start with **detection** and locate the relevant information using elements from the [Collection resource](/chronicle/docs/reference/rest/v1alpha/Collection) (/chronicle/docs/reference/rest/v1alpha/Collection).

Example: For example, suppose a detection rule produces the following information:

- Outcome variable: `dest_domain = "cymba1.com"`

- UDM field: `target.hostname = "cymba1.com"`

In the composite rule, you can access this data using the following paths:

- `detection.detection.outcomes["dest_domain"]` to access the `dest_domain` outcome variable.
- `detection.collection_elements.references.event.target.hostname` to access the `target.hostname` UDM field.
- `detection.time_window.start_time.seconds` to access the detection timestamp.

The Collection API and the `SecurityResult` API provide access to both:

- Detection metadata and outcome values (`detection.detection`)
- Underlying UDM events from referenced rules (`collection_elements`)

Reference detection content using rule ID or rule name

You can reference a rule by either its name or ID. We recommend this approach when your detection logic depends on specific rules. Referencing relevant rules by name or ID improves performance and prevents timeouts by reducing the data analyzed. For example, you can directly query fields like `target.url` or `principal.ip` from a known previous detection.

- **Reference a rule by rule ID (recommended):** use the `detection.detection.rule_id` field to reference a rule by ID. You can find the rule ID in the rule's URL in Google SecOps. User-generated rules have IDs in the format `ru_UUID`, while curated detections have IDs in the format `ur_UUID`. For example:

```
detection.detection.rule_id = "ru_e0d3f371-6832-4d20-b0ad-1f4e234acb2b"
```

- **Reference a rule by a rule name:** use the `detection.detection.rule_name` field to reference a rule by name. You can specify the exact rule name or use a regular expression to match it. For example:

- `detection.detection.rule_name = "My Rule Name"`
- `detection.detection.rule_name = "/PartOfName/"`

Note: We recommend referencing rule IDs because IDs are unique and don't change. Rule names can be modified, which can potentially break your composite detection.

Join your inputs in the `match` section

To connect related detections, events, or entities in a composite rule, define the `match` section using variables defined in the `events` section. These variables can include rule labels, outcome variables, match variables, detection fields, or collection elements.

For information about the syntax, see [Match section syntax](#) ([/chronicle/docs/detection/yara-l-2-0-syntax#match_section_syntax](#)).

Define the `condition` section

Define the `condition` section to evaluate the results of the `match` section. If the condition is `true`, an alert is generated. For information about the syntax, see [Condition section syntax](#) ([/chronicle/docs/detection/yara-l-2-0-syntax#condition_section_syntax](#)).

Use advanced techniques

This section explains how to apply advanced techniques while building composite rules.

Combine events and detections

Composite rules can combine multiple data sources, including UDM events, entity graph data, and detection fields. The following guidelines apply:

- **Use distinct variables per source:** Assign unique event variables to each data source (for example, `$e` for events, `$d` for detections), where the data source includes events, entities, and detections.
- **Join sources on shared context:** Connect data sources using common values, such as user IDs, IP addresses, or domain names in your rule's conditions.
- **Define a match window:** Always include a `match` clause with a time window no longer than 48 hours.

For example:

```
rule CheckCuratedDetection_with_EDR_and_EG {
  meta:
    author = "noone@cymbal.com"
  events:
    $d.detection.detection.rule_name = /SCC: Custom Modules: Configurable Bad
    $d.detection.collection_elements.references.event.network.dns.questions.n
```

```

$d.detection.collection_elements.references.event.principal.asset.hostname

$e.metadata.log_type = "LIMACHARLIE_EDR"
$e.metadata.product_event_type = "NETWORK_CONNECTIONS"
$domain = re.capture($e.principal.process.command_line, "\\s([a-zA-Z0-9.-
$hostname = re.capture($e.principal.hostname, "([^.]*)")

$prevalence.graph.metadata.entity_type = "DOMAIN_NAME"
$prevalence.graph.metadata.source_type = "DERIVED_CONTEXT"
$prevalence.graph.entity.hostname = $domain
$prevalence.graph.entity.domain.prevalence.day_count = 10
$prevalence.graph.entity.domain.prevalence.rolling_max <= 5
$prevalence.graph.entity.domain.prevalence.rolling_max > 0

match:
    $hostname over 1h

outcome:
    $risk_score = 80
    $CL_target = array($domain)

condition:
    $e and $d and $prevalence
}

```

Create sequential composite detections

Sequential composite detections identify patterns of related events where the sequence of detections is important, such as a brute-force login attempt detection, followed by a successful login. These patterns can combine multiple base detections, raw UDM events, or both

To create a sequential composite detection, you must enforce that order within your rule. To enforce the expected sequence, use one of the following methods:

- **Sliding windows:** Define the sequence of detections using sliding windows in your match conditions.
- **Timestamp comparisons:** Compare the timestamps of detections within your rule logic to verify that they happen in the selected order.

For example:

```
events:
  $d1.detection.detection.rule_name = "fileEvent_rule"
  $userid = $d1.detection.detection.outcomes["user"]
  $hostname = $d1.detection.detection.outcomes["hostname"]

  $d2.detection.detection.rule_name = "processExecution_rule"
  $userid = $d2.detection.detection.outcomes["user"]
  $hostname = $d2.detection.detection.outcomes["hostname"]

  $d3.detection.detection.rule_name = "networkEvent_rule"
  $userid = $d3.detection.detection.outcomes["user"]
  $hostname = $d3.detection.detection.outcomes["hostname"]

$d3.detection.collection_elements.references.event.metadata.event_timestamp.seconds

match:
  $userid over 24h after $d1
```

Risk scores

The `risk_score` is an integer from 0 to 100 that represents the potential severity or impact of a detection. A higher score indicates a more critical finding, assuming the detection is `true`.

To ensure consistency across the platform, we recommend using the following score ranges when assigning a `risk_score` to your custom detections. This alignment helps standardize alert prioritization and response workflows.

Severity	Score range	Description	Example
Alerting - Critical	90 - 100	Active compromise with the potential to have impact beyond a single user account or endpoint. Requires immediate review.	Mimikatz executed on Domain Controller.
Alerting - High	80 - 89	Active compromise of a single endpoint or entity. Should receive immediate review.	Production server calling out to recent, known C2.

Severity	Score range	Description	Example
Alerting - Medium	50 - 79	Potential security issue requiring investigation. No confirmed compromise but escalation is possible.	An exposed credential, no signs of misuse.
Non-Alerting - Low	20 - 49	Low-impact security event that, when combined with other indicators or observations, could lead to a more significant incident. No review generally needed, can be combined with other detections through composite rules to create an alert.	Internal port scan.
Non-Alerting Observations	1 - 19	Generally, information based detections intended to build situational awareness of a threat. Generally does not require review; can be combined with other detections through composite rules to generate alerts.	A login event, no signs of misuse.

Boolean expressions

Boolean expressions are expressions with a boolean type.

Comparisons

For a binary expression to use as condition, use the following syntax:

- `<EXPR> <OP> <EXPR>`

Expression can be either event field, variable, literal, or function expression.

For example:

- `$e.source.hostname = "host1234"`
- `$e.source.port < 1024`
- `1024 < $e.source.port`
- `$e1.source.hostname != $e2.target.hostname`
- `$e1.metadata.collected_timestamp.seconds > $e2.metadata.collected_timestamp.seconds`
- `$port >= 25`
- `$host = $e2.target.hostname`

- `"google-test" = strings.concat($e.principal.hostname, "-test")`
- `"email@google.org" = re.replace($e.network.email.from, "com", "org")`

If both sides are literals, it is regarded as a compilation error.

Functions

Some function expressions return boolean value, which can be used as an individual predicate in the `events` section. Such functions are:

- `re.regex()`
- `net.ip_in_range_cidr()`

For example:

- `re.regex($e.principal.hostname, `.*\.google\.com`)`
- `net.ip_in_range_cidr($e.principal.ip, "192.0.2.0/24")`

Reference list expressions

You can use reference lists in the `events` section. See the section on [Reference Lists](#) (`#reference_lists_syntax`) for more details.

Logical expressions

You can use the logical `and` and logical `or` operators in the `events` section as shown in the following examples:

- `$e.metadata.event_type = "NETWORK_DNS" or $e.metadata.event_type = "NETWORK_DHCP"`
- `($e.metadata.event_type = "NETWORK_DNS" and $e.principal.ip = "192.0.2.12") or ($e.metadata.event_type = "NETWORK_DHCP" and $e.principal.mac = "AB:CD:01:10:EF:22")`
- `not $e.metadata.event_type = "NETWORK_DNS"`

By default, the precedence order from highest to lowest is `not`, `and`, `or`.

For example, `"a or b and c"` is evaluated as `"a or (b and c)"` when the operators `or` and `and` are defined explicitly in the expression.

In the `events` section, predicates are joined using the `and` operator if an operator is not explicitly defined.

The order of evaluation may be different if the `and` operator is implied in the expression.

For example, consider the following comparison expressions where `or` is defined explicitly. The `and` operator is implied.

```
$e1.field = "bat"  
or $e1.field = "baz"  
$e2.field = "bar"
```

This example is interpreted as follows:

```
($e1.field = "bat" or $e1.field = "baz")  
and ($e2.field = "bar")
```

Because `or` is defined explicitly, the predicates surrounding `or` are grouped and evaluated first. The last predicate, `$e2.field = "bar"` is joined implicitly using `and`. The result is that order of evaluation changes.

Note: There is a limit on the number of `and` and `or` values you can specify for a single rule. This limit varies depending on the complexity of the rule and the complexity of the data in your Google SecOps account. Contact your Google SecOps representative for information on alternatives to this type of rule.

Enumerated types

You can use the operators with enumerated

(/chronicle/docs/reference/udm-field-list#event_enumerated_types) types. It can be applied to rules to simplify and optimize (use operator instead of reference lists) the performance.

In the following example, 'USER_UNCATEGORIZED' and 'USER_RESOURCE_DELETION' correspond to 15000 and 15014, so the rule will look for all the listed events:


```
$e.metadata.event_type >= "USER_CATEGORIZED" and $e.metadata.event_type <= "U
```

List of events:

- USER_RESOURCE_DELETION
- USER_RESOURCE_UPDATE_CONTENT
- USER_RESOURCE_UPDATE_PERMISSIONS
- USER_STATS
- USER_UNCATEGORIZED

Nocase Modifier

When you have a comparison expression between string values or a regular expression, you can append nocase at the end of the expression to ignore capitalization.

- `$e.principal.hostname != "http-server" nocase`
- `$e1.principal.hostname = $e2.target.hostname nocase`
- `$e.principal.hostname = /dns-server-[0-9]+/ nocase`
- `re.regex($e.target.hostname, `client-[0-9]+`) nocase`

This cannot be used when a type of field is an enumerated value. The following examples are invalid and will generate compilation errors:

- `$e.metadata.event_type = "NETWORK_DNS" nocase`
- `$e.network.ip_protocol = "TCP" nocase`

Repeated fields

In the Unified Data Model (UDM), some fields are labeled as repeated, which indicates that they are lists of values or other types of messages.

Repeated fields and boolean expressions

There are 2 kinds of boolean expressions that act on repeated fields:

1. Modified
2. Unmodified

Consider the following event:

```
event_original {  
  principal {  
    // ip is a repeated field  
    ip: [ "192.0.2.1", "192.0.2.2", "192.0.2.3" ]  
  
    hostname: "host"  
  }  
}
```

Modified expressions

The following sections describe the purpose and how to use the **any** and **all** modifiers in expressions.

any

If *any* element of the repeated field satisfies the condition, the event as a whole satisfies the condition.

- `event_original` satisfies `any $e.principal.ip = "192.0.2.1"`.
- `event_original` fails `any $e.repeated_field.field_a = "9.9.9.9"`.

all

If *all* elements of the repeated field satisfy the condition, the event as a whole satisfies the condition.

- `event_original` satisfies `net.ip_in_range_cidr(all $e.principal.ip, "192.0.2.0/8")`.
- `event_original` fails `all $e.principal.ip = "192.0.2.2"`.

Note: To use **any** or **all** with a function, the modifier must precede the repeated field and not the function. For example, `re.regex(any $e.about.hostname, `server-[0-9]+`)` is valid while `any re.regex($e.about.hostname, `server-[0-9]+`)` is not.

When writing a condition with **any** or **all**, be aware that negating the condition with **not** might not have the same meaning as using the negated operator.

For example:

- `not all $e.principal.ip = "192.168.12.16"` checks if not all IP addresses match 192.168.12.16, meaning the rule is checking whether at least one IP address does not match 192.168.12.16.
- `all $e.principal.ip != "192.168.12.16"` checks if all IP addresses don't match 192.168.12.16, meaning the rule is checking that no IP addresses match to 192.168.12.16.

Constraints:

- **any** and **all** operators are only compatible with repeated fields (not scalar fields).
- **any** and **all** cannot be used to join two repeated fields. For example, `any $e1.principal.ip = $e2.principal.ip` is not valid.
- **any** and **all** operators are not supported with the reference list expression.

Unmodified expressions

With unmodified expressions, each element in the repeated field is treated individually. If an event's repeated field contains n elements, then the rule is applied on n copies of the event, where each copy has one of the elements of the repeated field. These copies are transient and not stored.

The rule is applied on the following copies:

event copy	principal.ip	principal.hostname
event_copy_1	"192.0.2.1"	"host"
event_copy_2	"192.0.2.2"	"host"
event_copy_3	"192.0.2.3"	"host"

If *any* event copy satisfies *all* unmodified conditions on the repeated field, the event as a whole satisfies all the conditions. That means that if you have multiple conditions on a repeated field, then the event copy must satisfy *all* of them. The following rule examples use the preceding example dataset to demonstrate this behavior.

The following rule returns one match when run against the `event_original` example dataset, because `event_copy_1` satisfies all of the events predicates:

```
rule repeated_field_1 {
  meta:
  events:
    net.ip_in_range_cidr($e.principal.ip, "192.0.2.0/8") // Checks if IP address is in range
    $e.principal.ip = "192.0.2.1"
  condition:
    $e
}
```

The following rule doesn't return a match when run against the `event_original` example dataset, because there is no event copy in `$e.principal.ip` that satisfies *all* the event predicates.

```
rule repeated_field_2 {
  meta:
  events:
    $e.principal.ip = "192.0.2.1"
    $e.principal.ip = "192.0.2.2"
  condition:
    $e
}
```

Modified expressions on repeated fields are compatible with unmodified expressions on repeated fields because the element list is the same for each event copy. Consider the following rule:

```
rule repeated_field_3 {
  meta:
  events:
    any $e.principal.ip = "192.0.2.1"
```

```
    $e.principal.ip = "192.0.2.3"  
condition:  
    $e  
}
```

The rule is applied on the following copies:

event copy	principal.ip	any \$e.principal.ip
event_copy_1	"192.0.2.1"	["192.0.2.1", "192.0.2.2", "192.0.2.3"]
event_copy_2	"192.0.2.2"	["192.0.2.1", "192.0.2.2", "192.0.2.3"]
event_copy_3	"192.0.2.3"	["192.0.2.1", "192.0.2.2", "192.0.2.3"]

In this case, all copies satisfy `any $e.principal.ip = "192.0.2.1"` but only `event_copy_3` satisfies `$e.principal.ip = "192.0.2.3"`. As a result, the event as a whole would match.

Another way to think about these expression types are:

- Expressions on repeated fields which use `any` or `all` operate on the list in `event_original`.
- Expressions on repeated fields which don't use `any` or `all` operate on individual `event_copy_n` events.

Repeated fields and placeholders

Repeated fields work with placeholder assignments. Similar to unmodified expressions on repeated fields, a copy of the event is made for each element. Using the same example of `event_copy`, the placeholder takes the value of the `event_copy_n`'s repeated field value, for each of the event copies where *n* is the event copy number. If the placeholder is used in the match section, this can result in multiple matches.

The following example generates one match. The `$ip` placeholder is equal to `192.0.2.1` for `event_copy_1`, which satisfies the predicates in the rule. The match's event samples contain a single element, `event_original`.

```
// Generates 1 match.  
rule repeated_field_placeholder1 {  
    meta:
```

```

events:
  $ip = $e.principal.ip
  $ip = "192.0.2.1"
  $host = $e.principal.hostname

match:
  $host over 5m

condition:
  $e
}

```

The following example generates three matches. The `$ip` placeholder is equal to different values, for each of the different `event_copy_n` copies. The grouping is done on `$ip` since it is in the match section. Therefore, you get three matches where each match has a different value for the `$ip` match variable. Each match has the same event sample: a single element, `event_original`.

```

// Generates 3 matches.
rule repeated_field_placeholder2 {
  meta:
  events:
    $ip = $e.principal.ip
    net.ip_in_range_cidr($ip, "192.0.2.0/8") // Checks if IP matches 192.x.x.x

  match:
    $ip over 5m

  condition:
    $e
}

```

Note: `any` and `all` cannot be used when assigning a repeated field to a placeholder variable or joining with a field of another event. For example, `any $e.principal.ip = $ip` is not valid.

Outcomes using placeholders assigned to repeated fields

Placeholders are assigned to each *element* of each repeated field - not the entire list. Thus, when they're used in the outcome section, the outcome is calculated using only the elements that satisfied earlier sections.

Consider the following rule:

```
rule outcome_repeated_field_placeholder {
  meta:
  events:
    $ip = $e.principal.ip
    $ip = "192.0.2.1" or $ip = "192.0.2.2"
    $host = $e.principal.hostname

  match:
    $host over 5m

  outcome:
    $o = array_distinct($ip)

  condition:
    $e
}
```

There are 4 stages of execution for this rule. The first stage is event copying:

event copy	\$ip	\$host	\$e
event_copy_1	"192.0.2.1"	"host"	event_id
event_copy_2	"192.0.2.2"	"host"	event_id
event_copy_3	"192.0.2.3"	"host"	event_id

The events section will then filter out rows that don't match the filters:

event copy	\$ip	\$host	\$e
event_copy_1	"192.0.2.1"	"host"	event_id
event_copy_2	"192.0.2.2"	"host"	event_id

event_copy_3 is filtered out because "192.0.2.3" does not satisfy `$ip = "192.0.2.1"` or `$ip = "192.0.2.2"`.

The match section will then group by match variables and the outcome section will perform aggregation on each group:

\$host	\$o	\$e
"host"	["192.0.2.1", "192.0.2.2"]	event_id

`$o = array_distinct($ip)` is calculated using `$ip` from the previous stage and not the event copying stage.

Finally, the condition section will filter each group. Since this rule just checks for the existence of `$e`, the row from earlier will produce a single detection.

`$o` does not contain all the elements from `$e.principal.ip` because not all the elements satisfied all the conditions in the events section. However, all the elements of `e.principal.ip` will appear in the event sample because the event sample uses `event_original`.

Array indexing

You can perform array indexing on repeated fields. To access the *n*-th repeated field element, use the standard list syntax (elements are 0-indexed). An out-of-bounds element returns the default value.

- `$e.principal.ip[0] = "192.168.12.16"`
- `$e.principal.ip[999] = ""` If there are fewer than 1000 elements, this evaluates to `true`.

Constraints:

- An index must be a non-negative integer literal. For example, `$e.principal.ip[-1]` is not valid.
- Values that have an `int` type (for example, a placeholder set to `int`) don't count.
- Array indexing cannot be combined with `any` or `all`. For example, `any $e.intermediary.ip[0]` is not valid.
- Array indexing cannot be combined with `map` syntax. For example, `$e.additional.fields[0]["key"]` is not valid.
- If the field path contains multiple repeated fields, all repeated fields must use array indexing. For example, `$e.intermediary.ip[0]` is not valid because `intermediary` and `ip` are both repeated fields, but there is only an index for `ip`.

Repeated messages

When a message (<https://protobuf.dev/overview/#syntax>) field is repeated, an unintended effect is to reduce the likelihood of a match. This is illustrated in the following examples.

Consider the following event:

```
event_repeated_message {  
  // about is a repeated message field.  
  about {  
    // ip is a repeated string field.  
    ip: [ "192.0.2.1", "192.0.2.2", "192.0.2.3" ]  
  
    hostname: "alice"  
  }  
  about {  
    hostname: "bob"  
  }  
}
```

As stated for unmodified expressions on repeated fields, a temporary copy of the event is made for each element of the repeated field. Consider the following rule:

```
rule repeated_message_1 {  
  meta:  
  events:  
    $e.about.ip = "192.0.2.1"  
    $e.about.hostname = "bob"  
  condition:  
    $e  
}
```

The rule is applied on the following copies:

event copy	about.ip	about.hostname
event_copy_1	"192.0.2.1"	"alice"
event_copy_2	"192.0.2.2"	"alice"
event_copy_3	"192.0.2.3"	"alice"
event_copy_4	""	"bob"

The event does not match on the rule because there exists no event copy that satisfies all of the expressions.

Repeated messages and array indexing

Another unexpected behavior can occur when using array indexing with unmodified expressions on repeated message fields. Consider the following example rule which uses array indexing:

```
rule repeated_message_2 {
  meta:
  events:
    $e.about.ip = "192.0.2.1"
    $e.about[1].hostname = "bob"
  condition:
    $e
}
```

The rule is applied to the following copies:

event copy	about.ip	about[1].hostname
event_copy_1	"192.0.2.1"	"bob"
event_copy_2	"192.0.2.2"	"bob"
event_copy_3	"192.0.2.3"	"bob"
event_copy_4	""	"bob"

Since `event_copy_1` satisfies all of the expressions in `repeated_message_2`, the event matches on the rule.

This can lead to unexpected behavior because rule `repeated_message_1` lacked array indexing and produced no matches while rule `repeated_message_2` used array indexing and produced a match.

Comments

Designate comments with two slash characters (`// comment`) or multi-line comments set off using slash asterisk characters (`/* comment */`), as you would in C.

Literals

Nonnegative integers and floats, string, boolean, and regular expression literals are supported.

String and regular expression literals

You can use either of the following quotation characters to enclose strings in YARA-L 2.0. However, quoted text is interpreted differently depending on which one you use.

1. Double quotes (`"`) — Use for normal strings. Must include escape characters.
For example: `"hello\tworld"` — `\t` is interpreted as a tab
2. Back quotes (```) — Use to interpret all characters literally.
For example: ``hello\tworld`` — `\t` is not interpreted as a tab

For regular expressions, you have two options.

If you want to use regular expressions directly without the `re.regex()` function, use `/regex/` for the regular expression literals.

You can also use string literals as regular expression literals when you use the `re.regex()` function. Note that for double quote string literals, you must escape backslash characters with backslash characters, which can look awkward.

For example, the following regular expressions are equivalent:

- `re.regex($e.network.email.from, `.*altostrat\.com`)`
- `re.regex($e.network.email.from, ".*altostrat\\.com")`
- `$e.network.email.from = /.altostrat\.com/`

Google recommends using back quote characters for strings in regular expressions for ease of readability.

Operators

You can use the following operators in YARA-L:

Operator	Description
=	equal/declaration
!=	not equal
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal

Variables

In YARA-L 2.0, all variables are represented as `$<variable name>`.

You can define the following types of variables:

- **Event variables** — Represent groups of events in normalized form (UDM) or entity events. Specify conditions for event variables in the `events` section. You identify event variables using a name, event source, and event fields. Allowed sources are `udm` (for normalized events) and `graph` (for entity events). If the source is omitted, `udm` is set as the default source. Event fields are represented as a chain of `.<field name>` (for example, `$e.field1.field2`). Event field chains always start from the top-level source (UDM or Entity).
- **Match variables** — Declare in the `match` section. Match variables become grouping fields for the query, as one row is returned for each unique set of match variables (and for each time window). When the rule finds a match, the match variable values are returned. Specify what each match variable represents in the `events` section.
- **Placeholder variables** — Declare and define in the `events` section. Placeholder variables are similar to match variables. However, you can use placeholder variables in the `condition` section to specify match conditions.

Note: Every placeholder variable **must** be mapped to an event field. For example, if you only referenced the following placeholder in this single line in a rule, it would fail to compile since `$var` is not bound to an event variable: `$e.field != $var`

Use match variables and placeholder variables to declare relationships between event fields through transitive join conditions (see [Events Section Syntax](#) (`#events_section_syntax`) for more detail).

Keywords

Keywords in YARA-L 2.0 are case-insensitive. For example, `and` or `AND` are equivalent. Variable names must not conflict with keywords. For example, `$AND` or `$outcome` is invalid.

The following are keywords for detection engine rules: `rule`, `meta`, `match`, `over`, `events`, `condition`, `outcome`, `options`, `and`, `or`, `not`, `nocase`, `in`, `regex`, `cidr`, `before`, `after`, `all`, `any`, `if`, `max`, `min`, `sum`, `array`, `array_distinct`, `count`, `count_distinct`, `is`, and `null`.

Maps

YARA-L supports map access for Structs and Labels.

Structs and Labels

Some UDM fields use either the [Struct](#) (<https://developers.google.com/protocol-buffers/docs/reference/google.protobuf#struct>) or [Label](#) (</chronicle/docs/reference/udm-field-list#label>) data type.

To search for a specific key-value pair in both Struct and Label, use the standard map syntax:

```
// A Struct field.
$e.udm.additional.fields["pod_name"] = "kube-scheduler"
// A Label field.
$e.metadata.ingestion_labels["MetadataKeyDeletion"] = "startup-script"
```

The map access always returns a string.

Supported cases

Events and Outcome Section

```
// Using a Struct field in the events section
events:
    $e.udm.additional.fields["pod_name"] = "kube-scheduler"

// Using a Label field in the outcome section
outcome:
    $value = array_distinct($e.metadata.ingestion_labels["MetadataKeyDeletion"])
```

Assigning a map value to a Placeholder

```
$placeholder = $u1.metadata.ingestion_labels["MetadataKeyDeletion"]
```

Using a map field in a join condition

```
// using a Struct field in a join condition between two udm events $u1 and $u2
$u1.metadata.event_type = $u2.udm.additional.fields["pod_name"]
```

Unsupported cases

Maps are not supported in the following cases.

Combining any or all keywords with a map

For example, the following is not supported:

```
all $e.udm.additional.fields["pod_name"] = "kube-scheduler"
```

Other types of values

The map syntax can only return a string value. In the case of Struct (<https://developers.google.com/protocol-buffers/docs/reference/google.protobuf#struct>) data types,

the map syntax can only access keys whose values are strings. Accessing keys whose values are other primitive types like integers, is not possible.

Duplicate value handling

Map accesses always returns a single value. In the uncommon edge case that the map access could refer to multiple values, the map access will deterministically return the first value.

This can happen in either of the following cases:

- A label has a duplicate key.

The label structure represents a map, but does not enforce key uniqueness. By convention, a map should have unique keys, so Google SecOps does not recommend populating a label with duplicate keys.

The rule text `$e.metadata.ingestion_labels["dupe-key"]` would return the first possible value, `val1`, if run over the following data example:

```
// Disrecommended usage of label with a duplicate key:
event {
  metadata{
    ingestion_labels{
      key: "dupe-key"
      value: "val1" // This is the first possible value for "dupe-key"
    }
    ingestion_labels{
      key: "dupe-key"
      value: "val2"
    }
  }
}
```

- A label has an ancestor repeated field.

A repeated field might contain a label as a child field. Two different entries in the top-level repeated field might contain labels that have the same key. The rule text `$e.security_result.rule_labels["key"]` would return the first possible value, `val3`, if run over the following data example:

```

event {
  // security_result is a repeated field.
  security_result {
    threat_name: "threat1"
    rule_labels {
      key: "key"
      value: "val3" // This is the first possible value for "key"
    }
  }
  security_result {
    threat_name: "threat2"
    rule_labels {
      key: "key"
      value: "val4"
    }
  }
}

```

Access outcome variables in maps

This section explains how to access outcome variables within maps as their original data types (for example, integers, booleans, or lists of these types) rather than just strings. You can use this functionality for more flexibility and accuracy for your rule logic.

Outcome data is available in the following two fields:

- Outcome values retain their original types in the `variables` field.
- The `outcomes` field stores `string` versions for backward compatibility.

You can access these outcome values using the `variables` map to retrieve the specific type or access elements in a sequence using array indexing. You can either access a specific item in the sequence by its index or select the entire sequence to evaluate each value individually.

Syntax:

```
$d.detection.detection.variables[OUTCOME_NAME].TYPE_SUFFIX
```

Syntax for sequences:


```
$d.detection.detection.variables[OUTCOME_NAME].SEQUENCE_TYPE_SUFFIX.TYPE_
```

Examples:

- **Access a string outcome:**

```
$my_string_outcome = $d.detection.detection.variables["outcome_ip"].strir
```

This example retrieves the string value directly (for example, "1.1.1.1" if `outcome_ip` was a single string).

- **Access an integer outcome:**

```
$my_int_outcome = $d.detection.detection.variables["outcome_port"].int64_
```

This example retrieves the integer value (for example, 30).

- **Access a list of integers (Int64Sequence):**

```
$my_int_list = $d.detection.detection.variables["outcome_ports"].int64_se
```

This example retrieves the full list of integers and unnests them like repeated fields (for example, [2, 3, 4]).

- **Access a specific element from a list of integers:**

```
$first_int = $d.detection.detection.variables["outcome_ports"].int64_seq.
```

This example retrieves the first integer from the list (for example, 2).

- **Access a list of strings (StringSequence):**

```
$my_string_list = $d.detection.detection.variables["outcome_ips"].string_
```

This example retrieves the full list of strings and unnests them like repeated fields (for example, ["1.1.1.1", "2.2.2.2"]).

- **Access a specific element from a list of strings:**

```
$first_ip = $d.detection.detection.variables["outcome_ips"].string_seq.st
```

This example retrieves the first IP address from the list (for example, "1.1.1.1").

Available type suffixes for variables

For a full list of supported suffixes, see [FindingVariable](#) (/chronicle/docs/reference/rest/v1alpha/SecurityResult#findingvariable).

Note: For backward compatibility, the original **outcomes** field continues to return array outcomes as comma-separated strings in existing rules.

Functions

This section describes the YARA-L 2.0 functions that you can use in detection engine rules and search.

Note: The use of the event variable **\$e** is optional when YARA-L is used in search. Both **principal.hostname** and **\$e.principal.hostname** are supported in search.

These functions can be used in the following parts of a YARA-L rule:

- **events** section.
- **BOOL_CLAUSE** of a conditional in the [outcome section](#) (#outcome_section_syntax).

arrays.concat

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
arrays.concat(string_array, string_array)
```

Description

Returns a new string array by copying elements from original string arrays.

Param data types

ARRAY_STRINGS, ARRAY_STRINGS

Return type

ARRAY_STRINGS

Code samples

Example 1

The following example concatenates two different string arrays.

```
arrays.concat(["test1", "test2"], ["test3"]) = ["test1", "test2", "test3"]
```

Example 2

The following example concatenates arrays with empty string.

```
arrays.concat([""], [""]) = ["", ""]
```

Example 3

The following example concatenates empty arrays.

```
arrays.concat([], []) = []
```

arrays.join_string

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
arrays.join_string(array_of_strings, optional_delimiter)
```

Description

Converts an array of strings into a single string separated by the optional parameter. If no delimiter is provided, the empty string is used.

Param data types

ARRAY_STRINGS, STRING

Return type

STRING

Code samples

Here are some examples of how to use the function:

Example 1

This example joins an array with non-null elements and a delimiter.

```
arrays.join_string(["foo", "bar"], ",") = "foo,bar"
```

Example 2

This example joins an array with a null element and a delimiter.

```
arrays.join_string(["foo", NULL, "bar"], ",") = "foo,bar"
```

Example 3

This example joins an array with non-null elements and no delimiter.

```
arrays.join_string(["foo", "bar"]) = "foobar"
```

arrays.length

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
arrays.length(repeatedField)
```

Description

Returns the number of repeated field elements.

Param data types

LIST

Return type

NUMBER

Code samples

Example 1

Returns the number of repeated field elements.

```
arrays.length($e.principal.ip) = 2
```

Example 2

If multiple repeated fields are along the path, returns the total number of repeated field elements.

```
arrays.length($e.intermediary.ip) = 3
```

arrays.max

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
arrays.max(array_of_ints_or_floats)
```

Description

Returns the greatest element in an array or zero if the array is empty.

Param data types

ARRAY_INTS | ARRAY_FLOATS

Return type

FLOAT

Code samples

Here are some examples of how to use the function:

Example 1

This example returns the greater element in an array of integers.

```
arrays.max([10, 20]) = 20.000000
```

Example 2

This example returns the greater element in an array of floats.

```
arrays.max([10.000000, 20.000000]) = 20.000000
```

arrays.min

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
arrays.min(array_of_ints_or_floats[, ignore_zeros=false])
```

Description

Returns the smallest element in an array or zero if the array is empty. If the second, optional argument is set to true, elements equal to zero are ignored.

Param data types

ARRAY_INTS | ARRAY_FLOATS, BOOL

Return type

FLOAT

Code samples

Here are some examples of how to use the function:

Example 1

This example returns the smallest element in an array of integers.

```
arrays.min([10, 20]) = 10.000000
```

Example 2

This example returns the smallest element in an array of floats.

```
arrays.min([10.000000, 20.000000]) = 10.000000
```

Example 3

This example returns the smallest element in an array of floats, while ignoring the zeroes.

```
arrays.min([10.000000, 20.000000, 0.0], true) = 10.000000
```

`arrays.size`

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)
in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
arrays.size( array )
```

Description

Returns the size of the array. Returns 0 for an empty array.

Param data types

ARRAY_STRINGS | ARRAY_INTS | ARRAY_FLOATS

Return type

INT

Code samples

Example 1

This example uses a string array that contains two elements.

```
arrays.size([ "test1", "test2" ]) = 2
```

Example 2

This example uses an int array that contains 3 elements.

```
arrays.size([ 1, 2, 3 ]) = 3
```

Example 3

This example uses a float array that contains 1 element

```
arrays.size([1.200000]) = 1
```

Example 4

This example uses an empty array.

```
arrays.size([]) = 0
```

arrays.index_to_float

Supported [Rules](#) (/chronicle/docs/detection/default-rules)

in:

[Search](#) (/chronicle/docs/investigation/udm-search)

```
arrays.index_to_float(array, index)
```

Description

Returns the element at the given index of an array. The element at that index is returned as a float.

The index is an integer value which represents the position of an element in the array. By default, the first element of an array has an index of 0, and the last element has an index of n-1, where n is the size of the array. Negative indexing allows accessing array elements relative to the end of the array. For example, an index of -1 refers to the last element in the array and an index of -2 refers to the second to last element in the array.

Param data types

ARRAY_STRINGS | ARRAY_INTS | ARRAY_FLOATS, INT

Return type

FLOAT

Code samples

Example 1

The following example fetches an element at index 1 from an array of floats.

```
arrays.index_to_float([1.2, 2.1, 3.5, 4.6], 1) // 2.1
```

Example 2

The following example fetches an element at index -1 from an array of floats.

```
arrays.index_to_float([1.2, 2.1, 3.5, 4.6], 0-1) // 4.6
```

Example 3

The following example fetches an element for an index greater than the size of the array.

```
arrays.index_to_float([1.2, 2.1, 3.5, 4.6], 6) // 0.0
```

Example 4

The following example fetches an element from an empty array.

```
arrays.index_to_float([], 0) // 0.0
```

Example 5

The following example fetches an element at index 1 from a string array.

```
arrays.index_to_float(["1.2", "3.3", "2.4"], 1) // 3.3
```

Example 6

The following example fetches an element at index 2 from an array of integers.

```
arrays.index_to_float([1, 3, 2], 2) // 2.0
```

arrays.index_to_int

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
arrays.index_to_int(array_of_inputs, index)
```

Description

Returns the value at a given index in an array as an integer.

The index is an integer value which represents the position of an element in the array. By default, the first element of an array has an index of 0, and the last element has an index of n-1, where n is the size of the array. Negative indexing allows accessing array elements relative to the end of the array. For example, an index of -1 refers to the last element in the array and an index of -2 refers to the second to last element in the array.

Param data types

ARRAY_STRINGS|ARRAY_INTS|ARRAY_FLOATS, INT

Return type

INT

Code samples

Example 1

This function call returns 0 when the value at the index is a non-numeric string.

```
arrays.index_to_int(["str0", "str1", "str2"], 1) = 0
```

Example 2

This function returns the element at index -1.

```
arrays.index_to_int(["44", "11", "22", "33"], 0-1) = 33
```

Example 3

Returns 0 for the out-of-bounds element.

```
arrays.index_to_int(["44", "11", "22", "33"], 5) = 0
```

Example 4

This function fetches the element from the float array at index 1.

```
arrays.index_to_int([1.100000, 1.200000, 1.300000], 1) = 1
```

Example 5

This function fetches the element from the int array at index 0.

```
arrays.index_to_int([1, 2, 3], 0) = 1
```

arrays.index_to_str

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
arrays.index_to_str(array, index)
```

Description

Returns the element at the given index from the array as a string. The index is an integer value that represents the position of an element in the array. By default, the first element of an array has an index of 0, and the last element has an index of n-1, where n is the size of the array. Negative indexing allows accessing array elements from the end of the array. For example, an index of -1 refers to the last element in the array and an index of -2 refers to the second to last element in the array.

Param data types

ARRAY_STRINGS|ARRAY_INTS|ARRAY_FLOATS, INT

Return type

STRING

Code samples

Example 1

The following example fetches an element at index 1 from an array of strings.

```
arrays.index_to_str(["test1", "test2", "test3", "test4"], 1) // "test2"
```

Example 2

The following example fetches an element at index -1 (last element of the array) from an array of strings.

```
arrays.index_to_str(["test1", "test2", "test3", "test4"], 0-1) // "test4"
```

Example 3

The following example fetches an element for an index greater than the size of the array, which returns an empty string.

```
arrays.index_to_str(["test1", "test2", "test3", "test4"], 6) // ""
```

Example 4

The following example fetches an element from an empty array.

```
arrays.index_to_str([], 0) // ""
```

Example 5

The following example fetches an element at index 0 from an array of floats. The output is returned as a string.

```
arrays.index_to_str([1.200000, 3.300000, 2.400000], 0) // "1.2"
```

Example 6

The following example fetches an element at index 2 from an array of integers. The output is in the form of a string.

```
arrays.index_to_str([1, 3, 2], 2) // "2"
```

cast.as_bool

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
cast.as_bool(string_or_int)
```

Description

Function converts an int or string value into a bool value. Function calls with values that cannot be casted will return FALSE. Returns TRUE only for integer 1 and case insensitive string 'true'.

Param data types

INT | STRING

Return type

BOOL

Code samples

Example 1

This example shows how to cast a non-boolean string

```
cast.as_bool("123") = false
```


Example 2

Truthy integer (1)

```
cast.as_bool(1) = true
```

Example 3

Truthy string

```
cast.as_bool("true") = true
```

Example 4

Capital truthy string

```
cast.as_bool("TRUE") = true
```

Example 5

Negative integer

```
cast.as_bool(0-1) = false
```

Example 6

False integer (0)

```
cast.as_bool(0) = false
```

Example 7

empty string

```
cast.as_bool("") = false
```

cast.as_float

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
cast.as_float(string_to_cast)
```

Description

Converts a numeric string into a float. Any function calls with values that cannot be casted return 0. Floats maintain precision up to 7 decimal digits.

Param data types

STRING

Return type

FLOAT

Code samples

Example 1

Casting a non-numeric string returns 0.

```
cast.as_float("str") = 0.0000000
```

Example 2

Casting an empty string returns 0.

```
cast.as_float("") = 0.0000000
```

Example 3

Casting a valid numeric string returns a float value.

```
cast.as_float("1.012345678") = 1.0123456
```

cast.as_string

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
cast.as_string(int_or_bytes_or_bool, optional_default_string)
```

Description

The `cast.as_string` function transforms an `INT`, `BYTES`, or `BOOL` value into its string representation. You can provide an optional `default_string` argument to handle cases where the cast fails. If you omit the `default_string` argument, or if the input is an invalid UTF-8 or BASE64 byte sequence, the function returns an empty string.

Param data types

`INT | BYTES | BOOL, STRING`

Return type

`STRING`

Code samples

Integer to String Conversion

The function converts the integer 123 to the string "123".

```
cast.as_string(123) = "123"
```

Float to String Conversion

The function converts the float 2.25 to the string "2.25".

```
cast.as_string(2.25) = "2.25"
```

Bytes to String Conversion

The function converts the raw binary b'01 to the string "\x01".

```
cast.as_string(b'01, "") = "\x01"
```

Boolean to String Conversion

The function converts the boolean true to the string "true".

```
cast.as_string(true, "") = "true"
```

Failed Conversion (Defaults to the Optionally Provided String)

The function defaults to the string "casting error" when the value provided is invalid.

```
cast.as_string(9223372036854775808, "casting error") = "casting error"
```

fingerprint

Supported in: [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

```
hash.fingerprint2011(byteOrString)
```

Description

This function calculates the `fingerprint2011` hash of an input byte sequence or string. This function returns an unsigned `INT` value in the range `[2, 0xFFFFFFFFFFFFFFFF]`.

Note: This function shouldn't be used as a cryptographic secure hash.

Param data types

BTYE, STRING

Return type

INT

Code sample

```
id_fingerprint = hash.fingerprint2011("user123")
```

group

Supported in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
group(field1, field2, field3, ...)
```

Description

Group fields of a similar type into a placeholder variable.

In UDM search, grouped fields (/chronicle/docs/investigation/udm-search#search_grouped_fields) are used to search across multiple fields of a similar type. The group function is similar to grouped fields except that it lets you select which fields you want grouped together to trigger a detection. You can use the group function for gathering information about a specific entity (for example, a hostname, IP address, or userid) across different Noun types (/chronicle/docs/reference/udm-field-list#noun).

Note: For search, you can use grouped fields in the events section, but not in the match and outcome sections.

Code samples

Example 1

Group all the IP addresses together and provide a descending count of the most prevalent IP address in the time range scanned.

```
$ip = group(principal.ip, about.ip, target.ip)
$ip != ""
match:
  $ip
outcome:
  $count = count_distinct(metadata.id)
order:
  $count desc
```

hash.sha256

Supported in: Rules (/chronicle/docs/detection/default-rules)

```
hash.sha256(string)
```

Description

Returns a SHA-256 hash of the input string.

Param data types

STRING

Return type

STRING

Code samples

Example 1

This example shows the SHA-256 hash when the input is a valid string.

```
hash.sha256("str") = "8c25cb3686462e9a86d2883c5688a22fe738b0bbc85f458d2d2b5f3"
```

Example 2

This example shows the SHA-256 hash when the input is an empty string.

```
hash.sha256("") = "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b78"
```

math.abs

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
math.abs(numericExpression)
```

Description

Returns the absolute value of an integer or float expression.

Param data types

NUMBER

Return type

NUMBER

Code samples

Example 1

This example returns True if the event was more than 5 minutes from the time specified (in seconds from the Unix epoch), regardless of whether the event came before or after the time specified. A call to `math.abs` cannot depend on multiple variables or placeholders. For example, you cannot replace the hardcoded time value of 1643687343 in the following example with `$e2.metadata.event_timestamp.seconds`.

```
300 < math.abs($e1.metadata.event_timestamp.seconds - 1643687343)
```

math.ceil

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
math.ceil(number)
```


Description

Returns the smallest integer that is not less than the given number (rounding up). Will return 0 if the input is null or too big to fit in an int64.

Param data types

FLOAT

Return type

INT

Code samples

This section contains examples of using `math.ceil`.

Example 1

This example returns the ceil of a whole number.

```
math.ceil(2.000000) = 2
```

Example 2

This example returns the ceil of a negative number.

```
math.ceil(0-1.200000) = -1
```

Example 3

This example returns 0 as the ceil of a number that is too big for a 64 bit integer.

```
math.ceil(184467440737095516160.0) = 0
```

math.floor

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
math.floor(float_val)
```

Description

Returns the largest integer value that is not greater than the supplied value (rounding down). Returns 0 if the input is null or too large to fit into an int64.

Param data types

FLOAT

Return type

INT

Code samples

Example 1

This example shows a positive number case.

```
math.floor(1.234568) = 1
```

Example 2

This example shows a negative number case.

```
math.floor(0-1.234568) = -2
```

Example 3

This example shows a zero case.

```
math.floor(0.000000) = 0
```

math.geo_distance

Supported in: [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

```
math.geo_distance(longitude1, latitude1, longitude2, latitude2))
```

Description

Returns the distance between two geographic locations (coordinates) in meters. Returns -1 if the coordinates are invalid.

Parameter data types

FLOAT, FLOAT, FLOAT, FLOAT

Return type

FLOAT

Code samples

Example 1

The following example returns the distance when all parameters are valid coordinates:

```
math.geo_distance(-122.020287, 37.407574, -122.021810, 37.407574) = 134.564311
```

Example 2

The following example returns the distance when one of the parameters is a truncated coordinate:

```
math.geo_distance(-122.000000, 37.407574, -122.021810, 37.407574) = 1926.42196
```

Example 3

The following example returns -1 when one of the parameters is an invalid coordinate:

```
math.geo_distance(0-122.897680, 37.407574, 0-122.021810, 37.407574) = -1.000000
```

Example 4

The following example returns 0 when coordinates are the same:

```
math.geo_distance(-122.897680, 37.407574, -122.897680, 37.407574) = 0.000000
```

math.is_increasing

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
math.is_increasing(num1, num2, num3)
```

Description

Takes a list of numeric values (integers or doubles) and returns **True** if the values are in ascending order, and **False** otherwise.

Param data types

INT|FLOAT, INT|FLOAT, INT|FLOAT

Return type

BOOL

Code samples

Example 1

This example includes timestamp-like values in seconds.

```
math.is_increasing(1716769112, 1716769113, 1716769114) = true
```

Example 2

This example includes one negative double, one zero INT64, and one positive INT64 values.

```
math.is_increasing(-1.200000, 0, 3) = true
```

Example 3

This example includes one negative double, one zero INT64, and one negative INT64 values.

```
math.is_increasing(0-1.200000, 0, 0-3) = false
```

Example 4

This example includes two negative doubles and one zero INT64 value.

```
math.is_increasing(0-1.200000, 0-1.50000, 0) = false
```

Example 5

This example includes one negative double and two values that are the same.

```
math.is_increasing(0-1.200000, 0, 0) = false
```

math.log

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
math.log(numericExpression)
```

Description

Returns the natural log value of an integer or float expression.

Param data types

NUMBER

Return type

NUMBER

Code samples

Example 1

```
math.log($e1.network.sent_bytes) > 20
```

math.pow

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
math.pow(base, exponent)
```

Description

Returns the value of the first arg raised to the power of the second arg. Returns 0 in case of overflow.

Param data types

base: INT | FLOAT exponent: INT | FLOAT

Return type

FLOAT

Code samples

Example 1

This example shows an integer case.

```
math.pow(2, 2) // 4.00
```

Example 2

This example shows a fraction base case.

```
math.pow(2.200000, 3) // 10.648
```

Example 3

This example shows a fraction base and power case.

```
math.pow(2.200000, 1.200000) // 2.575771
```

Example 4

This example shows a negative power case.

```
math.pow(3, 0-3) // 0.037037
```

Example 5

This example shows a fraction power case.

```
math.pow(3, 0-1.200000) // 0.267581
```

Example 6

This example shows a negative base case.

```
math.pow(0-3, 0-3) // -0.037037
```

Example 7

This example shows a zero base case.

```
math.pow(0, 3) // 0
```

Example 8

This example shows a zero power case.

```
math.pow(9223372036854775807, 0) // 1
```

Example 9

This example shows a large base case.

```
math.pow(9223372036854775807, 1.200000) // 57262152889751593549824
```

math.random

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
math.random()
```

Description

Generates a pseudo-random value of type DOUBLE in the range of [0, 1), inclusive of 0 and exclusive of 1.

Return type

FLOAT

Code samples

The following example checks whether the random value is in the range [0, 1). none

```
if(math.random() >= 0 and math.random() < 1) = true
```

math.round

Supported in: [Search](#) (/chronicle/docs/investigation/udm-search)

```
math.round(numericExpression, decimalPlaces)
```

Description

Returns a value rounded to the nearest integer or to the specified number of decimal places.

Param data types

NUMBER

Return type

NUMBER

Code samples

```
math.round(10.7) // returns 11
math.round(1.2567, 2) // returns 1.25
math.round(0-10.7) // returns -11
math.round(0-1.2) // returns -1
math.round(4) // returns 4, math.round(integer) returns the integer
```

math.sqrt

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)
in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
math.sqrt(number)
```

Description

Returns the square root of the given number. Returns 0 in case of negative numbers.

Param data types

INT | FLOAT

Return type

FLOAT

Code samples

Example 1

This example returns the square root of an int argument.

```
math.sqrt(3) = 1.732051
```

Example 2

This example returns the square root of a negative int argument.

```
math.sqrt(-3) = 0.000000
```

Example 3

This example returns the square root of zero argument.

```
math.sqrt(0) = 0.000000
```

Example 4

This example returns the square root of a float argument.

```
math.sqrt(9.223372) = 3.037000
```

Example 5

This example returns the square root of a negative float argument.

```
math.sqrt(0-1.200000) = 0.000000
```

metrics

Supported in: [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

Metrics functions can aggregate large amounts of historical data. You can use this in your rule using `metrics.functionName()` in the outcome section.

For more information, see [YARA-L Metrics](/chronicle/docs/detection/metrics-functions) (/chronicle/docs/detection/metrics-functions).

net.ip_in_range_cidr

Supported in: [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
net.ip_in_range_cidr(ipAddress, subnetworkRange)
```

Description

Returns `true` when the given IP address is within the specified subnetwork.

You can use YARA-L to search for UDM events across all of the IP addresses within a subnetwork using the `net.ip_in_range_cidr()` statement. Both IPv4 and IPv6 are supported.

To search across a range of IP addresses, specify an IP UDM field and a CIDR range. YARA-L can handle both singular and repeating IP address fields.

To search across a range of IP addresses, specify an `ip` UDM field and a Classless Inter-Domain Routing (CIDR) range. YARA-L can handle both singular and repeating IP address fields.

Param data types

STRING, STRING

Return type

BOOL

Code samples

Example 1

IPv4 example:

```
net.ip_in_range_cidr($e.principal.ip, "192.0.2.0/24")
```

Example 2

IPv6 example:

```
net.ip_in_range_cidr($e.network.dhcp.yiaddr, "2001:db8::/32")
```

For an example rule using the `net.ip_in_range_cidr()` statement, see the example rule in [Single Event within Range of IP Addresses](#)

([/chronicle/docs/detection/yara-l-2-0-overview#single_event_within_range_of_ip_addresses](#)).

re.regex

Supported [Rules](#) ([/chronicle/docs/detection/default-rules](#))

in:

[Search](#) ([/chronicle/docs/investigation/udm-search](#))

You can define regular expression matching in YARA-L 2.0 using either of the following syntax:

- Using YARA-L syntax — Related to events. The following is a generic representation of this syntax:

```
$e.field = /regex/
```

- Using YARA-L syntax — As a function taking in the following parameters:
 - Field the regular expression is applied to.
 - Regular expression specified as a string.

The following is a generic representation of this syntax:

```
re.regex($e.field, `regex`)
```

Description

This function returns `true` if the string contains a substring that matches the regular expression provided. It is unnecessary to add `.*` to the beginning or at the end of the regular expression.

Notes

- To match the exact string or only a prefix or suffix, include the ^ (starting) and \$ (ending) anchor characters in the regular expression. For example, /^full\$/ matches "full" exactly, while /full/ could match "fullest", "lawfull", and "joyfully".
- If the UDM field includes newline characters, the `regexp` only matches the first line of the UDM field. To enforce full UDM field matching, add a (?s) to the regular expression. For example, replace /. *allUDM.*/ with /(?s). *allUDM.*/.
- You can use the `nocase` modifier after strings to indicate that the search should ignore capitalization.

Param data types

STRING, STRING

Param expression types

ANY, ANY

Return type

BOOL

Code samples

Example 1

```
// Equivalent to $e.principal.hostname = /google/  
re.regexp($e.principal.hostname, "google")
```

re.capture

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
re.capture(stringText, regex)
```

Description

Captures (extracts) data from a string using the regular expression pattern provided in the argument.

This function takes two arguments:

- **stringText**: the original string to search.
- **regex**: the regular expression indicating the pattern to search for.

The regular expression can contain 0 or 1 capture groups in parentheses. If the regular expression contains 0 capture groups, the function returns the first entire matching substring. If the regular expression contains 1 capture group, it returns the first matching substring for the capture group. Defining two or more capture groups returns a compiler error.

Param data types

STRING, STRING

Return type

STRING

Code samples

Example 1

In this example, if `$e.principal.hostname` contains "aaa1bbaa2" the following would be true, because the function returns the first instance. This example has no capture groups.

```
"aaa1" = re.capture($e.principal.hostname, "a+[1-9]")
```

Example 2

This example captures everything after the @ symbol in an email. If the `$e.network.email.from` field is `test@google.com`, the example returns `google.com`. The following example contains one capture group.


```
"google.com" = re.capture($e.network.email.from , "@(.*)")
```

Example 3

If the regular expression does not match any substring in the text, the function returns an empty string. You can omit events where no match occurs by excluding the empty string, which is especially important when you are using `re.capture()` with an inequality:

```
// Exclude the empty string to omit events where no match occurs.
"" != re.capture($e.network.email.from , "@(.*)")

// Exclude a specific string with an inequality.
"google.com" != re.capture($e.network.email.from , "@(.*)")
```

re.replace

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
re.replace(stringText, replaceRegex, replacementText)
```

Description

Performs a regular expression replacement.

This function takes three arguments:

- **stringText**: the original string.
- **replaceRegex**: the regular expression indicating the pattern to search for.
- **replacementText**: The text to insert into each match.

Returns a new string derived from the original **stringText**, where all substrings that match the pattern in **replaceRegex** are replaced with the value in **replacementText**. You can use

backslash-escaped digits (`\1` to `\9`) within `replacementText` to insert text matching the corresponding parenthesized group in the `replaceRegex` pattern. Use `\0` to refer to the entire matching text.

The function replaces non-overlapping matches and will prioritize replacing the first occurrence found. For example, `re.replace("banana", "ana", "111")` returns the string `"b111na"`.

Param data types

STRING, STRING, STRING

Return type

STRING

Code samples

Example 1

This example captures everything after the `@` symbol in an email, replaces `com` with `org`, and then returns the result. Notice the use of nested functions.

```
"email@google.org" = re.replace($e.network.email.from, "com", "org")
```

Example 2

This example uses backslash-escaped digits in the `replacementText` argument to reference matches to the `replaceRegex` pattern.

```
"test1.com.google" = re.replace(  
    $e.principal.hostname, // holds "test1.test2.google.com"  
    "test2\.[a-z]*\.[a-z]*",  
    "\\2.\\1" // \\1 holds "google", \\2 holds "com"  
)
```

Example 3

Note the following cases when dealing with empty strings and `re.replace()`:

Using empty string as `replaceRegex`:

```
// In the function call below, if $e.principal.hostname contains "name",  
// the result is: 1n1a1m1e1, because an empty string is found next to  
// every character in `stringText`.  
re.replace($e.principal.hostname, "", "1")
```

To replace an empty string, you can use `'^$',` as `replaceRegex`:

```
// In the function call below, if $e.principal.hostname contains the empty  
// string, "", the result is: "none".  
re.replace($e.principal.hostname, "'^$', "none")
```

sample_rate

Supported in: [Rules](#) (/chronicle/docs/detection/default-rules)

```
optimization.sample_rate(byteOrString, rateNumerator, rateDenominator)
```

Description

This function determines whether to include an event based on a deterministic sampling strategy. This function returns:

- `true` for a fraction of input values, equivalent to $(\text{rateNumerator} / \text{rateDenominator})$, indicating that the event should be included in the sample.
- `false` indicating that the event shouldn't be included in the sample.

This function is useful for optimization scenarios where you want to process only a subset of events. Equivalent to:

```
hash.fingerprint2011(byteOrString) % rateDenominator < rateNumerator
```

Param data types

- `byteOrString`: Expression that evaluates to either a `BYTE` or `STRING`.
- `rateNumerator`: `'INT'`
- `rateDenominator`: `'INT'`

Return type

`BOOL`

Code sample

```
events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $asset_id = $e.principal.asset.asset_id
    optimization.sample_rate($e.metadata.id, 1, 5) // Only 1 out of every 5 e

match:
    $asset_id over 1h

outcome:
    $event_count = count_distinct($e.metadata.id)
    // estimate the usage by multiplying by the inverse of the sample rate
    $usage_past_hour = sum(5.0 * $e.network.sent_bytes)

condition:
    // Requiring a certain number of events after sampling avoids bias (e.g. a
    // device with just 1 connection will still show up 20% of the time and
    // if we multiply that traffic by 5, we'll get an incorrect estimate)
    $e and ($usage_past_hour > 1000000000) and $event_count >= 100
```

`strings.base64_decode`

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)
in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.base64_decode(encodedString)
```

Description

Returns a string containing the base64 decoded version of the encoded string.

This function takes one base64 encoded string as an argument. If `encodedString` is not a valid base64 encoded string, the function returns `encodedString` unchanged.

Param data types

STRING

Return type

STRING

Code samples

Example 1

```
"test" = strings.base64_decode($e.principal.domain.name)
```

strings.coalesce

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)
in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.coalesce(a, b, c, ...)
```

Description

This function takes an unlimited number of arguments and returns the value of the first expression that does not evaluate to an empty string (for example, "non-zero value"). If all arguments evaluate to an empty string, the function call returns an empty string.

The arguments can be literals, event fields, or function calls. All arguments must be of **STRING** type. If any arguments are event fields, the attributes must be from the same event.

Param data types

STRING

Return type

STRING

Code samples

Example 1

The following example includes string variables as arguments. The condition evaluates to true when (1) `$e.network.email.from` is `suspicious@gmail.com` or (2) `$e.network.email.from` is empty and `$e.network.email.to` is `suspicious@gmail.com`.

```
"suspicious@gmail.com" = strings.coalesce($e.network.email.from, $e.network.er
```

Example 2

The following example calls the `coalesce` function with more than two arguments. This condition compares the first non-null IP address from event `$e` against values in the reference list `ip_watchlist`. The order that the arguments are coalesced in this call is the same as the order they are enumerated in the rule condition:

1. `$e.principal.ip` is evaluated first.
2. `$e.src.ip` is evaluated next.
3. `$e.target.ip` is evaluated next.

4. Finally, the string "No IP" is returned as a default value if the previous `ip` fields are unset.

```
strings.coalesce($e.principal.ip, $e.src.ip, $e.target.ip, "No IP") in %ip_wa
```

Example 3

The following example attempts to coalesce `principal.hostname` from event `$e1` and event `$e2`. It will return a compiler error because the arguments are different event variables.

```
// returns a compiler error
"test" = strings.coalesce($e1.principal.hostname, $e2.principal.hostname)
```

strings.concat

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.concat(a, b, c, ...)
```

Description

Returns the concatenation of an unlimited number of items, each of which can be a string, integer, or float.

If any arguments are event fields, the attributes must be from the same event.

Param data types

STRING, FLOAT, INT

Return type

STRING

Code samples

Example 1

The following example includes a string variable and integer variable as arguments. Both `principal.hostname` and `principal.port` are from the same event, `$e`, and are concatenated to return a string.

```
"google:80" = strings.concat($e.principal.hostname, ":", $e.principal.port)
```

Example 2

The following example includes a string variable and string literal as arguments.

```
"google-test" = strings.concat($e.principal.hostname, "-test") // Matches the
```

Example 3

The following example includes a string variable and float literal as arguments. When represented as strings, floats that are whole numbers are formatted without the decimal point (for example, 1.0 is represented as "1"). Additionally, floats that exceed sixteen decimal digits are truncated to the sixteenth decimal place.

```
"google2.5" = strings.concat($e.principal.hostname, 2.5)
```

Example 4

The following example includes a string variable, string literal, integer variable, and float literal as arguments. All variables are from the same event, `$e`, and are concatenated with the literals to return a string.


```
"google-test802.5" = strings.concat($e.principal.hostname, "-test", $e.princi
```

Example 5

The following example attempts to concatenate `principal.port` from event `$e1`, with `principal.hostname` from event `$e2`. It will return a compiler error because the arguments are different event variables.

```
// Will not compile
"test" = strings.concat($e1.principal.port, $e2.principal.hostname)
```

strings.contains

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.contains( str, substr )
```

Description

Returns true if a given string contains the specified substring. Otherwise it returns false.

Param data types

STRING, STRING

Return type

BOOL

Code samples

Example 1

This example returns true because the string has a substring "is".

```
strings.contains("thisisastring", "is") = true
```

Example 2

This example returns false because the string does not have substring "that".

```
strings.contains("thisisastring", "that") = false
```

strings.count_substrings

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.count_substrings(string_to_search_in, substring_to_count)
```

Description

When given a string and a substring, returns an int64 of the count of non-overlapping occurrences of the substring within the string.

Param data types

STRING, STRING

Return type

INT

Code samples

This section contains examples that calculate the number of times a substring appears in a given string.

Example 1

This example uses a non-null string and a non-null single substring character.

```
strings.count_substrings("this`string`has`four`backticks", "`") = 4
```

Example 2

This example uses a non-null string and a non-null substring greater than one character.

```
strings.count_substrings("str", "str") = 1
```

Example 3

This example uses a non-null string and an empty substring.

```
strings.count_substrings("str", "") = 0
```

Example 4

This example uses an empty string and a non-null substring greater than one character.

```
strings.count_substrings("", "str") = 0
```

Example 5

This example uses an empty string and an empty substring.

```
strings.count_substrings("", "") = 0
```

Example 6

This example uses a non-null string and a non-null substring that is greater than one character and greater than one occurrence.

```
strings.count_substrings("fooABABarABAbazABA", "AB") = 3
```

Example 7

This example uses a non-null string and a non-null substring that is greater than one character and greater than one occurrence. It highlights the limitation with overlapping substring occurrences

```
strings.count_substrings("ABABABA", "ABA") = 2
```

strings.extract_domain

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.extract_domain(url_string)
```

Description

Extracts the domain from a string.

Note: The function does not perform Unicode normalization.

Note: The public suffix data at publicsuffix.org also contains private domains. This function does not treat a private domain as a public suffix. For example, if `us.com` is a private domain in the public suffix data, `("foo.us.com")` returns `us.com` (the public suffix `com` plus the preceding label `us`) rather than `foo.us.com` (the private domain `us.com` plus the preceding label `foo`).

Note: The public suffix data might change over time. Consequently, input that produces default empty value now may produce a non-empty value in the future.

Param data types

STRING

Return type

STRING

Code samples

Example 1

This example shows an empty string

```
strings.extract_domain("") = ""
```

Example 2

random string, not a URL

```
strings.extract_domain("1234") = ""
```

Example 3

multiple backslashes

```
strings.extract_domain("\\\\") = ""
```

Example 4

non-alphabet characters handled gracefully

```
strings.extract_domain("http://例子.卷筒纸.中国") = "卷筒纸.中国"
```

Example 5

handling URIs

```
strings.extract_domain("mailto:?to=&subject=&body=") = ""
```

Example 6

multiple characters before actual URL

```
strings.extract_domain("      \t    !$5*^)&dahgsdfs;http://www.google.com") = "{
```

Example 7

special characters in URI #

```
strings.extract_domain("test#@google.com") = ""
```

Example 8

special characters in URL #

```
strings.extract_domain("https://test#@google.com") = ""
```

Example 9

positive test case

```
strings.extract_domain("https://google.co.in") = "google.co.in"
```

strings.extract_hostname

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.extract_hostname(string)
```

Description

Extracts the hostname from a string. This function is case sensitive.

Param data types

STRING

Return type

STRING

Code samples

Example 1

This example returns an empty string.

```
strings.extract_hostname("") = ""
```

Example 2

random string, not a URL

```
strings.extract_hostname("1234") = "1234"
```

Example 3

multiple backslashes

```
strings.extract_hostname("\\\\") = ""
```

Example 4

non-English characters handled gracefully

```
strings.extract_hostname("http://例子.卷筒纸.中国") = "例子.卷筒纸.中国"
```

Example 5

handling URIs

```
strings.extract_hostname("mailto:?to=&subject=&body=") = "mailto"
```

Example 6

multiple characters before actual URL

```
strings.extract_hostname("      \t    !$5*^)&dahgsdfs;http://www.google.com") =
```

Example 7

special characters in URI #

```
strings.extract_hostname("test#@google.com") = "test"
```

Example 8

special characters in URL #

```
strings.extract_hostname("https://test#@google.com") = "test"
```

strings.from_base64

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.from_base64(base64_encoded_string)
```

Description

Function converts a base64 encoded **STRING** value to a raw binary **BYTES** value. Function calls with values that cannot be casted return an empty **BYTES** by default.

Param data types

Return type

Code samples

Base64 Encoded String to Bytes Conversion

The function converts a base64 encoded string to its raw binary bytes representation.

```
strings.from_base64("AAAAAG+OxVhtAm+d2sVuny/hW4oAAAAAAQAAAM0AAAA=") = b'000000'
```

Failed Conversion (Defaults to Empty Bytes)

The function defaults to empty bytes if the provided value is invalid.

```
strings.from_base64("invalid-value") = b'
```

strings.from_hex

Supported Rules (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.from_hex(hex_string)
```

Description

Returns the bytes associated with the given hex string.

Param data types

STRING

Return type

BYTES

Code samples

Get bytes associated with a given hex string.

Example 1

This example shows non-hex character conversions.

```
strings.from_hex("str") // returns empty bytes
```

Example 2

This example shows input with empty string.

```
strings.from_hex("") // returns empty bytes
```

Example 3

This example shows hex string conversion.

```
strings.from_hex("1234") // returns 1234 bytes
```

Example 4

This example shows non-ASCII characters conversion.

```
strings.from_hex("筒纸.中国") // returns empty bytes
```

strings.ltrim

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.ltrim(string_to_trim, cutset)
```

Description

Trims leading white spaces from a given string. This function removes leading characters present in that cutset.

Param data types

STRING, STRING

Return type

STRING

Code samples

The following are example use cases.

Example 1

This example uses the same first and second argument.

```
strings.ltrim("str", "str") = ""
```

Example 2

This example uses an empty string as the second argument.

```
strings.ltrim("str", "") = "str"
```

Example 3

This example uses an empty string as the first argument, and a string as the second argument.

```
strings.ltrim("", "str") = ""
```

Example 4

This example uses strings that contain white spaces, and a string as the second argument.

```
strings.ltrim("a aastraa aa ", " a") = "straa aa "
```

strings.reverse

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.reverse(String)
```

Description

Returns a string that is the reverse of the input string.

Param data types

STRING

Return type

STRING

Code samples

Example 1

The following example passes a short string.

```
strings.reverse("str") = "rts" // The function returns 'rts'.
```

Example 2

The following example passes an empty string.

```
strings.reverse("") = ""
```

Example 3

The following example passes a palindrome.

```
strings.reverse("tacocat") = "tacocat"
```

strings.rtrim

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.rtrim(string_to_trim, cutset)
```

Description

Trims trailing white spaces from a given string. Removes trailing characters that are present in that cutset.

Param data types

STRING, STRING

Return type

STRING

Code samples

The following are example use cases.

Example 1

The following example passes the same string as the first and second argument.

```
strings.rtrim("str", "str") = ""
```

Example 2

The following example passes an empty string as the second argument.

```
strings.rtrim("str", "") = "str"
```

Example 3

The following example passes an empty string as the first argument and a non-empty string as the second argument.

```
strings.rtrim("", "str") = ""
```

Example 4

The following example passes a string containing white spaces as the first argument and a non-empty string as the second argument.

```
strings.rtrim("a aastraa aa ", " a") = "a aastr"
```

strings.to_lower

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.to_lower(stringText)
```

Description

This function takes an input string and returns a string after changing all characters to lowercase

Param data types

STRING

Return type

STRING

Code samples

Example 1

The following example returns `true`.

```
"test@google.com" = strings.to_lower($e.network.email.to)
```

strings.to_upper

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.to_upper(string_val)
```

Description

Returns the original string with all alphabetic characters in uppercase.

Param data types

STRING

Return type

STRING

Code samples

Example 1

The following example returns the supplied argument in uppercase.

```
strings.to_upper("example") = "EXAMPLE"
```

strings.trim

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.trim(string_to_trim, cutset)
```

Description

Trims leading and trailing white spaces from a given string. Also, remove unwanted characters (specified by the cutset argument) from the input string.

Param data types

STRING, STRING

Return type

STRING

Code samples

The following are example use cases.

Example 1

In the following example, the same string is passed as the input string and the cutset, which results in an empty string.

```
strings.trim("str", "str") // ""
```

Example 2

In the following example, an empty string is passed as the cutset, which results in the original string `str` because there are no characters specified in the cutset to remove.

```
strings.trim("str", "") = "str"
```

Example 3

In the following example, the function yields an empty string because the input string is already empty and there are no characters to remove.

```
strings.trim("", "str") = ""
```

Example 4

In the following example, the function yields `str` because the trim function removes the following:

- trailing whitespace in `"a aastraa aa "`
- the characters specified in the cutset (space, a)

```
strings.trim("a aastraa aa ", " a") = "str"
```

strings.url_decode

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
strings.url_decode(url_string)
```

Description

Given a URL string, decode the escape characters and handle UTF-8 characters that have been encoded. Returns empty string if decoding fails.

Param data types

STRING

Return type

STRING

Code samples

Example 1

This example shows a positive test case.

```
strings.url_decode("three%20nine%20four") = "three nine four"
```

Example 2

This example shows an empty string case.

```
strings.url_decode("") // ""
```

Example 3

This example shows non-alphabet characters handling.

```
strings.url_decode("%E4%B8%8A%E6%B5%B7%2B%E4%B8%AD%E5%9C%8B") // "上海+中國"
```

Example 4

This example shows a sample URL decoding.

timestamp.as_unix_seconds

in:

Description

117/149

Param data types

STRING, STRING

Return type

INT

Code samples

Example 1

Valid epoch timestamp

```
timestamp.as_unix_seconds("2024-02-22 10:43:00") = 1708598580
```

Example 2

Valid epoch timestamp with the America/New_York time zone

```
timestamp.as_unix_seconds("2024-02-22 10:43:00", "America/New_York") = 1708610
```

timestamp.current_seconds

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
timestamp.current_seconds()
```

Description

Returns an integer representing the current time in Unix seconds. This is approximately equal to the detection timestamp and is based on when the rule is run. This function is a

synonym of the function `timestamp.now()`.

Param data types

NONE

Return type

INT

Code samples

Example 1

The following example returns `true` if the certificate has been expired for more than 24 hours. It calculates the time difference by subtracting the current Unix seconds, and then comparing using a greater than operator.

```
86400 < timestamp.current_seconds() - $e.network.tls.certificate.not_after
```

timestamp.get_date

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
timestamp.get_date(unix_seconds [, time_zone])
```

Description

This function returns a string in the format `YYYY-MM-DD`, representing the day a timestamp is in.

- `unix_seconds` is an integer representing the number of seconds past Unix epoch, such as `$e.metadata.event_timestamp.seconds`, or a placeholder containing that value.

- `time_zone` is optional and is a string representing a time_zone. If omitted, the default is "GMT". You can specify time zones using string literals. The options are:
 - The TZ database name, for example "America/Los_Angeles". For more information, see the ["TZ Database Name" column from this page](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones) (https://en.wikipedia.org/wiki/List_of_tz_database_time_zones)
 - The time zone offset from UTC, in the format `(+|-)H[H][[:M[M]]]`, for example: "-08:00".

Here are examples of valid `time_zone` specifiers, which you can pass as the second argument to time extraction functions:

"America/Los_Angeles", or "-08:00". ("PST" is not supported)
"America/New_York", or "-05:00". ("EST" is not supported)
"Europe/London"
"UTC"
"GMT"

Param data types

INT, STRING

Return type

STRING

Code samples

Example 1

In this example, the `time_zone` argument is omitted, so it defaults to "GMT".

```
$ts = $e.metadata.collected_timestamp.seconds
```

```
timestamp.get_date($ts) = "2024-02-19"
```

Example 2

This example uses a string literal to define the `time_zone`.

```
$ts = $e.metadata.collected_timestamp.seconds  
  
timestamp.get_date($ts, "America/Los_Angeles") = "2024-02-20"
```

timestamp.get_minute

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
timestamp.get_minute(unix_seconds [, time_zone])
```

Description

This function returns an integer in the range `[0, 59]` representing the minute.

- `unix_seconds` is an integer representing the number of seconds past Unix epoch, such as `$e.metadata.event_timestamp.seconds`, or a placeholder containing that value.
- `time_zone` is optional and is a string representing a time zone. If omitted, the default is "GMT". You can specify time zones using string literals. The options are:
 - The TZ database name, for example "America/Los_Angeles". For more information, see the ["TZ Database Name" column from this page](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones) (https://en.wikipedia.org/wiki/List_of_tz_database_time_zones)
 - The time zone offset from UTC, in the format `(+|-)H[H][:M[M]]`, for example: "-08:00".

Here are examples of valid `time_zone` specifiers, which you can pass as the second argument to time extraction functions:

```
"America/Los_Angeles", or "-08:00". ("PST" is not supported)  
"America/New_York", or "-05:00". ("EST" is not supported)
```

```
"Europe/London"  
"UTC"  
"GMT"
```

Param data types

INT, STRING

Return type

INT

Code samples

Example 1

In this example, the `time_zone` argument is omitted, so it defaults to "GMT".

```
$ts = $e.metadata.collected_timestamp.seconds  
  
timestamp.get_hour($ts) = 15
```

Example 2

This example uses a string literal to define the `time_zone`.

```
$ts = $e.metadata.collected_timestamp.seconds  
  
timestamp.get_hour($ts, "America/Los_Angeles") = 15
```

timestamp.get_hour

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
timestamp.get_hour(unix_seconds [, time_zone])
```

Description

This function returns an integer in the range [0, 23] representing the hour.

- **unix_seconds** is an integer representing the number of seconds past Unix epoch, such as `$e.metadata.event_timestamp.seconds`, or a placeholder containing that value.
- **time_zone** is optional and is a string representing a time zone. If omitted, the default is "GMT". You can specify time zones using string literals. The options are:
 - The TZ database name, for example "America/Los_Angeles". For more information, see the ["TZ Database Name" column from this page](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones) (https://en.wikipedia.org/wiki/List_of_tz_database_time_zones)
 - The time zone offset from UTC, in the format (+|-)H[H][:M[M]], for example: "-08:00".

Here are examples of valid **time_zone** specifiers, which you can pass as the second argument to time extraction functions:

```
"America/Los_Angeles", or "-08:00". ("PST" is not supported)
"America/New_York", or "-05:00". ("EST" is not supported)
"Europe/London"
"UTC"
"GMT"
```

Param data types

INT, STRING

Return type

INT

Code samples

Example 1

In this example, the `time_zone` argument is omitted, so it defaults to "GMT".

```
$ts = $e.metadata.collected_timestamp.seconds  
timestamp.get_hour($ts) = 15
```

Example 2

This example uses a string literal to define the `time_zone`.

```
$ts = $e.metadata.collected_timestamp.seconds  
timestamp.get_hour($ts, "America/Los_Angeles") = 15
```

timestamp.get_day_of_week

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
timestamp.get_day_of_week(unix_seconds [, time_zone])
```

Description

This function returns an integer in the range [1, 7] representing the day of week starting with Sunday. For example, 1 = Sunday and 2 = Monday.

- `unix_seconds` is an integer representing the number of seconds past Unix epoch, such as `$e.metadata.event_timestamp.seconds`, or a placeholder containing that value.
- `time_zone` is optional and is a string representing a `time_zone`. If omitted, the default is "GMT". You can specify time zones using string literals. The options are:

- The TZ database name, for example "America/Los_Angeles". For more information, see the ["TZ Database Name" column from this page](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones) (https://en.wikipedia.org/wiki/List_of_tz_database_time_zones)
- The time zone offset from UTC, in the format `(+|-)H[H][:M[M]]`, for example: `"-08:00"`.

Here are examples of valid `time_zone` specifiers, which you can pass as the second argument to time extraction functions:

```
"America/Los_Angeles", or "-08:00". ("PST" is not supported)
"America/New_York", or "-05:00". ("EST" is not supported)
"Europe/London"
"UTC"
"GMT"
```

Param data types

INT, STRING

Return type

INT

Code samples

Example 1

In this example, the `time_zone` argument is omitted, so it defaults to "GMT".

```
$ts = $e.metadata.collected_timestamp.seconds

timestamp.get_day_of_week($ts) = 6
```

Example 2

This example uses a string literal to define the `time_zone`.

```
$ts = $e.metadata.collected_timestamp.seconds

timestamp.get_day_of_week($ts, "America/Los_Angeles") = 6
```

timestamp.get_timestamp

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
timestamp.get_timestamp(unix_seconds, optional timestamp_format/time_granular:
```

Description

This function returns a string in the format YYYY-MM-DD, representing the day a timestamp is in.

- `unix_seconds` is an integer representing the number of seconds past Unix epoch, such as `$e.metadata.event_timestamp.seconds`, or a placeholder containing that value.
- `timestamp_format` is optional and is a string representing the format for the timestamp. If omitted, the default is `%F %T`. You can specify the format using a date time format string or one of the following time granularity: `SECOND`, `MINUTE`, `HOURL`, `DATE`, `WEEK`, `MONTH`, or `YEAR`. For more formatting options, see [Format elements for date and time parts](/bigquery/docs/reference/standard-sql/format-elements#format_elements_date_time) (/bigquery/docs/reference/standard-sql/format-elements#format_elements_date_time)
- `time_zone` is optional and is a string representing a time zone. If omitted, the default is `GMT`. You can specify time zones using string literals. The options are as follows:
 - The IANA Time Zone (TZ) database name, for example, `America/Los_Angeles`. For more information, see the [list of tz database time zones on Wikipedia](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones) (https://en.wikipedia.org/wiki/List_of_tz_database_time_zones).
 - The time zone offset from UTC, in the format `(+|-)H[H][:M[M]]`, for example: `"-08:00"`.

Here are examples of valid `time_zone` specifiers, which you can pass as the second argument to time extraction functions:

```
"America/Los_Angeles", or "-08:00". ("PST" is not supported)
"America/New_York", or "-05:00". ("EST" is not supported)
"Europe/London"
"UTC"
"GMT"
```

Param data types

INT, STRING, STRING

Return type

STRING

Code samples

Example 1

In this example, the `time_zone` argument is omitted, so it defaults to GMT.

```
$ts = $e.metadata.collected_timestamp.seconds
timestamp.get_timestamp($ts) = "2024-02-22 10:43:51"
```

Example 2

This example uses a string literal to define the `time_zone`.

```
$ts = $e.metadata.collected_timestamp.seconds
timestamp.get_timestamp($ts, "%F %T", "America/Los_Angeles") = "2024-02-22 10
```

Example 3

This example uses a string literal to define the `timestamp_format`.

```
$ts = $e.metadata.collected_timestamp.seconds  
  
timestamp.get_timestamp($ts, "%Y-%m", "GMT") = "2024-02"
```

Example 4

This example formats a unix timestamp as a string at second granularity.

```
timestamp.get_timestamp(1708598631, "SECOND", "GMT") = "2024-02-22 10:43:51"
```

Example 5

This example formats a unix timestamp as a string at minute granularity.

```
timestamp.get_timestamp(1708598631, "MINUTE", "GMT") = "2024-02-22 10:43"
```

Example 6

This example formats a unix timestamp as a string at hour granularity.

```
timestamp.get_timestamp(1708598631, "HOUR", "GMT") = "2024-02-22 10"
```

Example 7

This example formats a unix timestamp as a string at day granularity.

```
timestamp.get_timestamp(1708598631, "DATE", "GMT") = "2024-02-22"
```


Example 8

This example formats a unix timestamp as a string at week granularity.

```
timestamp.get_timestamp(1708598631, "WEEK", "GMT") = "2024-02-18"
```

Example 9

This example formats a unix timestamp as a string at month granularity.

```
timestamp.get_timestamp(1708598631, "MONTH", "GMT") = "2024-02"
```

Example 10

This example formats a unix timestamp as a string at year granularity.

```
timestamp.get_timestamp(1708598631, "YEAR", "GMT") = "2024"
```

timestamp.get_week

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in:

[Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
timestamp.get_week(unix_seconds [, time_zone])
```

Description

This function returns an integer in the range [0, 53] representing the week of the year. Weeks begin with Sunday. Dates before the first Sunday of the year are in week 0.

- `unix_seconds` is an integer representing the number of seconds past Unix epoch, such as `$e.metadata.event_timestamp.seconds`, or a placeholder containing that value.
- `time_zone` is optional and is a string representing a time zone. If omitted, the default is "GMT". You can specify time zones using string literals. The options are:
 - The TZ database name, for example "America/Los_Angeles". For more information, see the ["TZ Database Name" column from this page](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones) (https://en.wikipedia.org/wiki/List_of_tz_database_time_zones)
 - The time zone offset from UTC, in the format `(+|-)H[H][:M[M]]`, for example: `"-08:00"`.

Here are examples of valid `time_zone` specifiers, which you can pass as the second argument to time extraction functions:

```
"America/Los_Angeles", or "-08:00". ("PST" is not supported)
"America/New_York", or "-05:00". ("EST" is not supported)
"Europe/London"
"UTC"
"GMT"
```

Param data types

INT, STRING

Return type

INT

Code samples

Example 1

In this example, the `time_zone` argument is omitted, so it defaults to "GMT".

```
$ts = $e.metadata.collected_timestamp.seconds  
  
timestamp.get_week($ts) = 0
```

Example 2

This example uses a string literal to define the `time_zone`.

```
$ts = $e.metadata.collected_timestamp.seconds  
  
timestamp.get_week($ts, "America/Los_Angeles") = 0
```

timestamp.now

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)
in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

```
timestamp.now()
```

Description

Returns the number of seconds since 1970-01-01 00:00:00 UTC. This is also known as *Unix epoch time*.

Return type

INT

Code samples

Example 1

The following example returns a timestamp for code executed on May 22, 2024 at 18:16:59.

```
timestamp.now() = 1716401819 // Unix epoch time in seconds for May 22, 2024 at
```

window.avg

Supported in: [Rules](#) (/chronicle/docs/detection/default-rules)

```
window.avg(numeric_values [, should_ignore_zero_values])
```

Description

Returns the average of the input values (which can be Integers or Floats). Setting the optional second argument to true ignores zero values.

Param data types

INT | FLOAT

Return type

FLOAT

Code samples

Example 1

This example shows the integer average.

```
// This rule sets the outcome $size_mode to the average
// file size in the 5 minute match window.
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
```

```
$size_mode = window.avg($e.file.size) // yields 2.5 if the event file size is
```

Example 2

This example shows the float average.

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $size_mode = window.avg($e.file.size) // yields 1.75 if the event file size is
```

Example 3

Negative input average

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $size_mode = window.avg($e.file.size) // yields 0.6 if the event file size is
```

Example 4

0 returns 0

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $size_mode = window.avg($e.file.size) // yields 0 if the event file size is
```

Example 5

Ignoring 0 values

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $size_mode = window.avg($e.file.size, true) // yields 394 if the event file
```

window.first

Supported in: [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

```
window.first(values_to_sort_by, values_to_return)
```

Description

This aggregation function returns a string value derived from an event with the lowest correlated int value in the match window. An example use case is getting the userid from the event with the lowest timestamp in the match window (earliest event).

Param data types

INT, STRING

Return type

STRING

Code samples

Get a string value derived from an event with the lowest correlated int value in the match window.

```
// This rule sets the outcome $first_event to the lowest correlated int value
// in the 5 minute match window.
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $first_event = window.first($e.metadata.timestamp.seconds, $e.metadata.event
```

window.last

Supported in: [Rules](#) (/chronicle/docs/detection/default-rules)

```
window.last(values_to_sort_by, values_to_return)
```

Description

This aggregation function returns a string value derived from an event with the highest correlated int value in the match window. An example use case is getting the userid from the event with the lowest timestamp in the match window (highest timestamp).

Param data types

INT, STRING

Return type

STRING

Code samples

Get a string value derived from an event with the highest correlated int value in the match window.

```
// This rule sets the outcome $last_event to the highest correlated int value
// in the 5 minute match window.
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $last_event = window.first($e.metadata.timestamp.seconds, $e.metadata.event.
```

window.median

Supported in: [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

```
window.median(numeric_values, should_ignore_zero_values)
```

Description

Return the median of the input values. If there are 2 median values, only 1 will be non-deterministically chosen as the return value.

Param data types

INT | FLOAT, BOOL

Return type

FLOAT

Code samples

Example 1

This example returns the median when the input values aren't zero.


```
rule median_file_size {
  meta:
  events:
    $e.metadata.event_type = "FILE_COPY"
    $userid = $e.principal.user.userid
  match:
    $userid over 1h
  outcome:
    $median_file_size = window.median($e.principal.file.size) // returns 2 :
  condition:
    $e
}
```

Example 2

This example returns the median when the input includes some zero values that shouldn't be ignored.

```
rule median_file_size {
  meta:
  events:
    $e.metadata.event_type = "FILE_COPY"
    $userid = $e.principal.user.userid
  match:
    $userid over 1h
  outcome:
    $median_file_size = window.median($e.principal.file.size) // returns 1 :
  condition:
    $e
}
```

Example 3

This example returns the median when the input includes some zero values which should be ignored.

```
rule median_file_size {
  meta:
  events:
    $e.metadata.event_type = "FILE_COPY"
    $userid = $e.principal.user.userid
  match:
    $userid over 1h
  outcome:
    $median_file_size = window.median($e.principal.file.size, true) // return
  condition:
    $e
}
```

Example 4

This example returns the median when the input includes all zero values which should be ignored.

```
rule median_file_size {
  meta:
  events:
    $e.metadata.event_type = "FILE_COPY"
    $userid = $e.principal.user.userid
  match:
    $userid over 1h
  outcome:
    $median_file_size = window.median($e.principal.file.size) // returns 0 :
  condition:
    $e
}
```

Example 5

This example shows that, when there are multiple medians, only one median is returned.

```
rule median_file_size {
  meta:
  events:
```

```
    $e.metadata.event_type = "FILE_COPY"
    $userid = $e.principal.user.userid
  match:
    $userid over 1h
  outcome:
    $median_file_size = window.median($e.principal.file.size) // returns 1 :
  condition:
    $e
}
```

window.mode

Supported in: Rules (/chronicle/docs/detection/default-rules)

window.mode(values)

Description

Return the mode of the input values. In case of multiple possible mode values, only one of those values will be non-deterministically chosen as the return value.

Param data types

INT | FLOAT | STRING

Return type

STRING

Code samples

Example 1

Get mode of the values in the match window.

```
// This rule sets the outcome $size_mode to the most frequently occurring
// file size in the 5 minute match window.
```

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $size_mode = window.mode($e.file.size) // yields 1.6 if the event file size
```

window.stddev

Supported in: [Rules](#) (/chronicle/docs/detection/default-rules)

```
window.stddev(numeric_values)
```

Description

Returns the standard deviation of input values in a match window.

Param data types

INT|FLOAT

Return type

FLOAT

Code samples

Example 1

This example returns the standard deviation of integers in a match window.

```
// This rule creates a detection when the file size stddev in 5 minutes for a
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
```

```
$p1 = window.stddev($e.file.size) // yields 4.0 if the event file size value  
condition:  
$e and #p1 > 2
```

Example 2

This example returns the standard deviation of floats in a match window.

```
events:  
$e.user.userid = $userid  
match:  
$userid over 5m  
outcome:  
$p1 = window.stddev($e.file.size) // yields 4.488686 if the event file size  
condition:  
$e and #p1 > 2
```

Example 3

This example returns the standard deviation in a match window that contains negative numbers.

```
events:  
$e.user.userid = $userid  
match:  
$userid over 5m  
outcome:  
$p1 = window.stddev($e.file.size) // yields 48.644972 if the event file size  
condition:  
$e and #p1 > 2
```

Example 4

This example returns with zero standard deviation when all values in the match window are the same.

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $p1 = window.stddev($e.file.size) // yields 0.000000 if the event file size
condition:
  $e and #p1 > 2
```

Example 5

This example returns the standard deviation of a match window containing positive and negative numbers.

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $p1 = window.stddev($e.file.size) // yields 1.000000 if the event file size
condition:
  $e and #p1 > 10
```

window.variance

Supported in: [Rules](#) (/chronicle/docs/detection/default-rules)

`window.variance(values)`

Description

This function returns the specified variance of the input values.

Param data types

INT|FLOAT

Return type

FLOAT

Code samples

Example 1

This example returns the variance of all integers.

```
// This rule creates a detection when the file size variance in 5 minutes for
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $p1 = window.variance($e.file.size) // yields 16 if the event file size val
condition:
  $e and #p1 > 10
```

Example 2

This example returns the variance of all floats.

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $p1 = window.variance($e.file.size) // yields 20.148300 if the event file s:
condition:
  $e and #p1 > 10
```

Example 3

This example returns the variance of negative numbers.

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $p1 = window.variance($e.file.size) // yields 2366.333333 if the event file
condition:
  $e and #p1 > 10
```

Example 4

This example returns a small variance value.

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $p1 = window.variance($e.file.size) // yields 0.000000 if the event file si
condition:
  $e and #p1 > 10
```

Example 5

This example returns a zero variance.

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $p1 = window.variance($e.file.size) // yields 0.000000 if the event file si
condition:
  $e and #p1 > 10
```


Example 6

This example returns the variance of positive and negative numbers.

```
events:
  $e.user.userid = $userid
match:
  $userid over 5m
outcome:
  $p1 = window.variance($e.file.size) // yields 1.000000 if the event file si
condition:
  $e and #p1 > 10
```

bytes.to_base64

Supported [Rules](/chronicle/docs/detection/default-rules) (/chronicle/docs/detection/default-rules)

in: [Search](/chronicle/docs/investigation/udm-search) (/chronicle/docs/investigation/udm-search)

`bytes.to_base64(bytes, optional_default_string)`

Description

Function converts a `bytes` value to a `base64 encoded string`. Function calls with values that cannot be casted return an empty string by default.

Param data types

BYTES, STRING

Return type

STRING

Code samples

Raw Binary Bytes to Base64 Encoded String

The function converts the raw binary bytes to base64 encoded string.

```
bytes.to_base64(b'000000006f8ec5586d026f9ddac56e9f2fe15b8a000000001000000cd00')
```

Failed Conversion (Defaults to the Optionally Provided String)

The function defaults to the "invalid bytes" when the bytes value provided isn't valid.

```
bytes.to_base64(b'000000006f8ec5586d', "invalid bytes") = "invalid bytes"
```

Function to placeholder assignment

You can assign the result of a function call to a placeholder in the `events` section. For example:

```
$placeholder = strings.concat($e.principal.hostname, "my-string").
```

You can then use the placeholder variables in the `match`, `condition`, and `outcome` sections. However, there are two limitations with function to placeholder assignment:

1. Every placeholder in function to placeholder assignment must be assigned to an expression containing an event field. For example, the following examples are valid:

```
$ph1 = $e.principal.hostname
$ph2 = $e.src.hostname
```

```
// Both $ph1 and $ph2 have been assigned to an expression containing an event field
$ph1 = strings.concat($ph2, ".com")
```

```
$ph1 = $e.network.email.from
$ph2 = strings.concat($e.principal.hostname, "@gmail.com")
```

```
// Both $ph1 and $ph2 have been assigned to an expression containing an event  
$ph1 = strings.to_lower($ph2)
```

However, the following example is invalid:

```
$ph1 = strings.concat($e.principal.hostname, "foo")  
$ph2 = strings.concat($ph1, "bar") // $ph2 has NOT been assigned to an event
```

2. Function call should depend on **one and exactly one** event. However, more than one field from the same event can be used in function call arguments. For example, the following is valid:

```
$ph = strings.concat($event.principal.hostname, "string2")  
  
$ph = strings.concat($event.principal.hostname, $event.src.hostname)
```

However, the following is invalid:

```
$ph = strings.concat("string1", "string2")  
  
$ph = strings.concat($event.principal.hostname,  
$anotherEvent.src.hostname)
```

Reference Lists syntax

See our [page on Reference Lists](#)

(<https://cloud.google.com/chronicle/docs/reference/reference-lists.md>) for more information on reference list behavior and reference list syntax.

You can use reference lists in the **events** or **outcome** sections. Here is the syntax for using various types of reference lists in a rule:

```
// STRING reference list  
$e.principal.hostname in %string_reference_list  
  
// REGEX reference list  
$e.principal.hostname in regex %regex_reference_list  
  
// CIDR reference list
```

```
$e.principal.ip in cidr %cidr_reference_list
```

You can also use the `not` operator and the `nocase` operator with reference lists as shown in the following example:

```
// Exclude events whose hostnames match substrings in my_regex_list.  
not $e.principal.hostname in regex %my_regex_list  
  
// Event hostnames must match at least 1 string in my_string_list (case insens  
$e.principal.hostname in %my_string_list nocase
```

The `nocase` operator is compatible with `STRING` lists and `REGEX` lists.

For performance reasons, the Detection Engine restricts reference list usage.

- Maximum `in` statements in a rule, with or without special operators: 7
- Maximum `in` statements with the `regex` operator: 4
- Maximum `in` statements with the `cidr` operator: 2

Type checking

Google SecOps performs type checking against your YARA-L syntax as you create rules within the interface. The type checking errors displayed help you to revise the rule in such a way as to confirm that it functions correctly.

The following are examples of **invalid** predicates:

```
// $e.target.port is of type integer which cannot be compared to a string.  
$e.target.port = "80"  
  
// "LOGIN" is not a valid event_type enum value.  
$e.metadata.event_type = "LOGIN"
```

Detection Event Sampling

Detections from multi-event rules contain event samples to provide context about the events that caused the detection. There is a limit of up to 10 event samples for each event variable defined in the rule. For example, if a rule defines 2 event variables, each detection can have up to 20 event samples. The limit applies to each event variable separately. If one event variable has 2 applicable events in this detection, and the other event variable has 15 applicable events, the resulting detection contains 12 event samples (2 + 10).

Any event samples over the limit are omitted from the detection.

If you want more information about the events that caused your detection, you can use aggregations in the [outcome section](#) (#outcome_section_syntax) to output additional information in your detection.

If you are viewing detections in the UI, you can download all events samples for a detection. For more information, see [Download events](#) (/chronicle/docs/detection/downloading-events).

Need more help? [Get answers from Community members and Google SecOps professionals.](#) (<https://security.googlecloudcommunity.com/google-security-operations-2>)

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](#) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](#) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](#) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-08-25 UTC.